

Código explicado linha por linha

Use este apêndice como uma conversa ao lado do editor. Comece pelo arquivo que você está estudando, acompanhe o trecho original e leia a explicação na mesma linha.

As linhas de decisão explicam o comportamento. As de estrutura explicam a sintaxe e a rotina em que aparecem. Os comentários originais preservam os créditos e observações do tutorial; na tela, podem ser mostrados pelo controle abaixo. Na impressão, eles ficam ocultos para concentrar a consulta nas linhas ativas.

O núcleo documentado inclui agentes, ambiente, ações internas, testes, lançador e configurações. Bibliotecas, Gradle Wrapper, arquivos gerados e geradores editoriais não fazem parte desta leitura por linha.

[Guia introdutório em PDF](#) · [Este apêndice em PDF](#) · [Laboratório no GitHub](#)

- | | |
|---|---|
| 1. src/agt/dummy.asl | 13. src/env/mining/WorldView.java |
| 2. src/agt/leader.asl | 14. src/test/java/mining/ModelChecks.java |
| 3. src/agt/miner-coleta.asl | 15. src/test/java/mining/ValidationProbe.java |
| 4. src/agt/miner.asl | 16. scripts/laboratorio.py |
| 5. src/test/agt/verificacao.asl | 17. scripts/indicadores.py |
| 6. src/agt/jia/dist.java | 18. scripts/verificar_indicadores.py |
| 7. src/agt/jia/get_direction.java | 19. build.gradle |
| 8. src/agt/jia/neighbour.java | 20. settings.gradle |
| 9. src/agt/jia/random.java | 21. gold_miners.jcm |
| 10. src/agt/jia/random_direction.java | 22. configuracoes/verificacao.jcm |
| 11. src/env/mining/MiningPlanet.java | 23. laboratorio.cmd |
| 12. src/env/mining/WorldModel.java | 24. .devcontainer/devcontainer.json |
| | 25. logging.properties |

src/agt/dummy.asl

Mantém um participante sem estratégia ativa de mineração. Ajuda a estudar um minerador de cada vez.

[Abrir arquivo original](#) · 19 linhas · numeração da cópia documentada

Linha	Código	Como ler
3	<code>{ include("\$jacamoJar/templates/common-cartago.asl") }</code>	Inclui os planos comuns de integração Jason–CARTaGO fornecidos pelo JaCaMo.
15	<code>!start.</code>	Solicita o subobjetivo de start. O ponto e vírgula liga a próxima etapa do corpo.
19	<code>+!start : true <- .wait(500); skip; !start.</code>	Oferece um plano para start. O trecho após os dois-pontos é o contexto de seleção.

src/agt/leader.asl

Mantém um placar baseado em mensagens dropped. Os mineradores desta versão ainda não enviam essas mensagens; o total do ambiente é uma informação diferente.

[Abrir arquivo original](#) · 24 linhas · numeração da cópia documentada

Linha	Código	Como ler
3	<code>{ include("\$jacamoJar/templates/common-cartago.asl") }</code>	Inclui os planos comuns de integração Jason–CartAgO fornecidos pelo JaCaMo.
11	<code>score(miner1,0).</code>	Inicializa o placar que o líder manterá a partir das mensagens recebidas; ainda não é o placar global do ambiente.
12	<code>score(miner2,0).</code>	Inicializa o placar que o líder manterá a partir das mensagens recebidas; ainda não é o placar global do ambiente.
13	<code>score(miner3,0).</code>	Inicializa o placar que o líder manterá a partir das mensagens recebidas; ainda não é o placar global do ambiente.
14	<code>score(miner4,0).</code>	Inicializa o placar que o líder manterá a partir das mensagens recebidas; ainda não é o placar global do ambiente.
20	<code>+dropped[source(A)] : score(A,S)</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
21	<code><- -score(A,S);</code>	Retira a crença indicada, porque ela deixou de representar a situação que o agente quer manter.
22	<code>+score(A,S+1);</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
23	<code>-dropped[source(A)];</code>	Retira a crença indicada, porque ela deixou de representar a situação que o agente quer manter.
24	<code>.print("Agent ",A," has dropped ",S+1," pieces of gold").</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.

src/agt/miner-coleta.asl

Acrescenta memória das descobertas, escolha de alvo, coleta e depósito. O depósito 0,0 e a consulta global de caminhos ainda são limitações do exemplo.

[Abrir arquivo original](#) · 219 linhas · numeração da cópia documentada

Linha	Código	Como ler
3	<code>{ include("\$jacamoJar/templates/common-cartago.asl") }</code>	Inclui os planos comuns de integração Jason–CartAgO fornecidos pelo JaCaMo.
11	<code>last_dir(null). // the last movement I did</code>	Compõe o plano ou a crença deste trecho de AgentSpeak. Leia em conjunto com o evento acima e a condição que permite adotá-lo.
12	<code>free.</code>	Crença inicial de disponibilidade para explorar ou assumir uma coleta.
28	<code>+free : gsize(_,W,H) & jia.random(RX,W-1) & jia.random(RY,H-1)</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
29	<code><- .print("I am going to go near (",RX,"",RY,"");</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
30	<code>!go_near(RX,RY).</code>	Solicita o subobjetivo de iniciar a exploração em direção a um destino escolhido. O ponto e vírgula liga a próxima etapa do corpo.

Linha	Código	Como ler
31	<code>+free // gsize is unknown yet</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
32	<code><- .wait(100); ++free.</code>	Espera o intervalo indicado antes de repetir a observação. A espera evita uma repetição imediata sem novas percepções.
38	<code>+near(X,Y) : free <- ++free.</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
52	<code>+!go_near(X,Y) : free</code>	Oferece um plano para iniciar a exploração em direção a um destino escolhido. O trecho após os dois-pontos é o contexto de seleção.
53	<code><- -near(_,_);</code>	Retira a crença indicada, porque ela deixou de representar a situação que o agente quer manter.
54	<code>-last_dir(_);</code>	Retira a crença indicada, porque ela deixou de representar a situação que o agente quer manter.
55	<code>!near(X,Y).</code>	Solicita o subobjetivo de chegar à vizinhança do destino ou tratar uma rota não encontrada. O ponto e vírgula liga a próxima etapa do corpo.
59	<code>+!near(X,Y) : (pos(AgX,AgY) & jia.neighbour(AgX,AgY,X,Y))</code>	Oferece um plano para chegar à vizinhança do destino ou tratar uma rota não encontrada. O trecho após os dois-pontos é o contexto de seleção.
60	<code><- .print("I am at ", "(" ,AgX,"", AgY,")", " which is near (" ,X,"", Y,")");</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
61	<code>+near(X,Y).</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
65	<code>+!near(X,Y) : pos(AgX,AgY) & last_dir(skip)</code>	Oferece um plano para chegar à vizinhança do destino ou tratar uma rota não encontrada. O trecho após os dois-pontos é o contexto de seleção.
66	<code><- .print("I am at ", "(" ,AgX,"", AgY,")", " and I can't get to' (" ,X,"", Y,")");</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
67	<code>+near(X,Y).</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
69	<code>+!near(X,Y) : not near(X,Y)</code>	Oferece um plano para chegar à vizinhança do destino ou tratar uma rota não encontrada. O trecho após os dois-pontos é o contexto de seleção.
70	<code><- !next_step(X,Y);</code>	Solicita o subobjetivo de escolher e executar um único movimento. O ponto e vírgula liga a próxima etapa do corpo.
71	<code>!near(X,Y).</code>	Solicita o subobjetivo de chegar à vizinhança do destino ou tratar uma rota não encontrada. O ponto e vírgula liga a próxima etapa do corpo.
72	<code>+!near(X,Y) : true</code>	Oferece um plano para chegar à vizinhança do destino ou tratar uma rota não encontrada. O trecho após os dois-pontos é o contexto de seleção.
73	<code><- !near(X,Y).</code>	Solicita o subobjetivo de chegar à vizinhança do destino ou tratar uma rota não encontrada. O ponto e vírgula liga a próxima etapa do corpo.

Linha	Código	Como ler
81	<code>+!next_step(X,Y) : pos(AgX,AgY) // I already know my position</code>	Oferece um plano para escolher e executar um único movimento. O trecho após os dois-pontos é o contexto de seleção.
82	<code><- jia.get_direction(AgX, AgY, X, Y, D);</code>	Pede à ação interna uma direção para o alvo. A implementação usa busca A* e ainda consulta o mapa global.
83	<code>--last_dir(D);</code>	Substitui a crença indicada por seu novo valor, gerando o evento correspondente à atualização.
84	<code>D.</code>	Executa como ação a direção que foi unificada com a variável D.
85	<code>+!next_step(X,Y) : not pos(_,_) // I still do not know my position</code>	Oferece um plano para escolher e executar um único movimento. O trecho após os dois-pontos é o contexto de seleção.
86	<code><- !next_step(X,Y).</code>	Solicita o subobjetivo de escolher e executar um único movimento. O ponto e vírgula liga a próxima etapa do corpo.
87	<code>-!next_step(X,Y) : true // failure handling -> start again!</code>	Trata a falha do objetivo de escolher e executar um único movimento. O trecho após os dois-pontos é o contexto de seleção.
88	<code><- --last_dir(null);</code>	Substitui a crença indicada por seu novo valor, gerando o evento correspondente à atualização.
89	<code>!next_step(X,Y).</code>	Solicita o subobjetivo de escolher e executar um único movimento. O ponto e vírgula liga a próxima etapa do corpo.
97	<code>+!pos(X,Y) : pos(X,Y)</code>	Oferece um plano para chegar exatamente à posição informada. O trecho após os dois-pontos é o contexto de seleção.
98	<code><- .print("I've reached ",X,"x",Y).</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
99	<code>+!pos(X,Y) : not pos(X,Y)</code>	Oferece um plano para chegar exatamente à posição informada. O trecho após os dois-pontos é o contexto de seleção.
100	<code><- !next_step(X,Y);</code>	Solicita o subobjetivo de escolher e executar um único movimento. O ponto e vírgula liga a próxima etapa do corpo.
101	<code>!pos(X,Y).</code>	Solicita o subobjetivo de chegar exatamente à posição informada. O ponto e vírgula liga a próxima etapa do corpo.
115	<code>+cell(X,Y,gold) <- +gold(X,Y).</code>	Transforma a percepção de ouro em memória própria. Assim a descoberta não some apenas por sair do campo de visão, mas pode ficar desatualizada.
117	<code>@pgold[atomic] // atomic: so as not to handle another event until handle gold is initialised</code>	Identifica ou configura o plano. Em planos atomic, a atomicidade é local ao agente, sem garantir exclusão entre mineradores.
118	<code>+gold(X,Y)</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
119	<code>: not carrying_gold & free</code>	Exige este contexto para selecionar o plano; & combina condições e not consulta a ausência de uma solução na base de crenças.
120	<code><- -free;</code>	Retira a crença indicada, porque ela deixou de representar a situação que o agente quer manter.
121	<code>.print("Gold perceived: ",gold(X,Y));</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.

Linha	Código	Como ler
122	<code>!init_handle(gold(X,Y)).</code>	Solicita o subobjetivo de preparar a intenção de coleta e abandonar a exploração incompatível. O ponto e vírgula liga a próxima etapa do corpo.
137	<code>@pih1[atomic]</code>	Identifica ou configura o plano. Em planos atomic, a atomicidade é local ao agente, sem garantir exclusão entre mineradores.
138	<code>+!init_handle(Gold)</code>	Oferece um plano para preparar a intenção de coleta e abandonar a exploração incompatível.
139	<code>: .desire(near(_,_))</code>	Exige este contexto para selecionar o plano; & combina condições e not consulta a ausência de uma solução na base de crenças.
140	<code><- .print("Dropping near(_,_) desires and intentions to handle ",Gold);</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
141	<code>.drop_desire(near(_,_));</code>	Abandona o objetivo indicado e suas intenções, abrindo espaço para a tarefa de coleta.
142	<code>!init_handle(Gold).</code>	Solicita o subobjetivo de preparar a intenção de coleta e abandonar a exploração incompatível. O ponto e vírgula liga a próxima etapa do corpo.
143	<code>@pih2[atomic]</code>	Identifica ou configura o plano. Em planos atomic, a atomicidade é local ao agente, sem garantir exclusão entre mineradores.
144	<code>+!init_handle(Gold)</code>	Oferece um plano para preparar a intenção de coleta e abandonar a exploração incompatível.
145	<code>: pos(X,Y)</code>	Exige este contexto para selecionar o plano; & combina condições e not consulta a ausência de uma solução na base de crenças.
146	<code><- .print("Going for ",Gold);</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
147	<code>!!handle(Gold). // must use !! to perform "handle" as not atomic</code>	Inicia o objetivo indicado numa intenção separada, permitindo sair da preparação atômica da coleta.
149	<code>+!handle(gold(X,Y))</code>	Oferece um plano para acompanhar o ciclo de coleta e depósito do alvo.
150	<code>: not free</code>	Exige este contexto para selecionar o plano; & combina condições e not consulta a ausência de uma solução na base de crenças.
151	<code><- .print("Handling ",gold(X,Y)," now.");</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
152	<code>!pos(X,Y);</code>	Solicita o subobjetivo de chegar exatamente à posição informada. O ponto e vírgula liga a próxima etapa do corpo.
153	<code>!ensure(pick,gold(X,Y));</code>	Solicita o subobjetivo de executar a operação de coleta ou depósito e conferir sua condição. O ponto e vírgula liga a próxima etapa do corpo.
154	<code>!pos(0,0);</code>	Solicita retorno ao depósito fixo em 0,0. Este é um dos pontos a generalizar para outros cenários.
155	<code>!ensure(drop, 0);</code>	Solicita o subobjetivo de executar a operação de coleta ou depósito e conferir sua condição. O ponto e vírgula liga a próxima etapa do corpo.

Linha	Código	Como ler
156	<code>.print("Finish handling ",gold(X,Y));</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
157	<code>!!choose_gold.</code>	Inicia o objetivo indicado numa intenção separada, permitindo sair da preparação atômica da coleta.
160	<code>-!handle(G) : G</code>	Trata a falha do objetivo de acompanhar o ciclo de coleta e depósito do alvo. O trecho após os dois-pontos é o contexto de seleção.
161	<code><- .print("failed to catch gold ",G);</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
162	<code>.abolish(G); // ignore source</code>	Remove as crenças que correspondem ao padrão, inclusive quando possuem anotações de origem.
163	<code>!!choose_gold.</code>	Inicia o objetivo indicado numa intenção separada, permitindo sair da preparação atômica da coleta.
164	<code>-!handle(G) : true</code>	Trata a falha do objetivo de acompanhar o ciclo de coleta e depósito do alvo. O trecho após os dois-pontos é o contexto de seleção.
165	<code><- .print("failed to handle ",G," , it isn't in the BB anyway");</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
166	<code>!!choose_gold.</code>	Inicia o objetivo indicado numa intenção separada, permitindo sair da preparação atômica da coleta.
170	<code>+!ensure(pick,_) : pos(X,Y) & gold(X,Y)</code>	Oferece um plano para executar a operação de coleta ou depósito e conferir sua condição. O trecho após os dois-pontos é o contexto de seleção.
171	<code><- pick;</code>	Solicita a operação pick ao artefato em foco. Os efeitos serão observados pelo agente.
172	<code>?carrying_gold;</code>	Confere a crença de carga após a coleta. Se ela não puder ser satisfeita, o plano falha.
173	<code>-gold(X,Y).</code>	Retira a crença indicada, porque ela deixou de representar a situação que o agente quer manter.
177	<code>+!ensure(drop, _) : carrying_gold & pos(0,0)</code>	Oferece um plano para executar a operação de coleta ou depósito e conferir sua condição. O trecho após os dois-pontos é o contexto de seleção.
178	<code><- drop.</code>	Solicita a operação drop ao artefato em foco. Os efeitos serão observados pelo agente.
186	<code>+!choose_gold</code>	Oferece um plano para escolher outro ouro conhecido ou voltar à exploração.
187	<code>: not gold(_,_)</code>	Exige este contexto para selecionar o plano; & combina condições e not consulta a ausência de uma solução na base de crenças.
188	<code><- -+free.</code>	Substitui a crença indicada por seu novo valor, gerando o evento correspondente à atualização.
192	<code>+!choose_gold</code>	Oferece um plano para escolher outro ouro conhecido ou voltar à exploração.
193	<code>: gold(_,_)</code>	Exige este contexto para selecionar o plano; & combina condições e not consulta a ausência de uma solução na base de crenças.

Linha	Código	Como ler
194	<code><- .findall(gold(X,Y),gold(X,Y),LG);</code>	Reúne todos os alvos conhecidos que satisfazem a consulta numa lista para comparação.
195	<code>!calc_gold_distance(LG,LD);</code>	Solicita o subobjetivo de montar a lista de distâncias dos alvos conhecidos. O ponto e vírgula liga a próxima etapa do corpo.
196	<code>.length(LD,LLD); LLD > 0;</code>	Obtém o tamanho da lista e verifica se há candidatos antes de selecionar um alvo.
197	<code>.print("Gold distances: ",LD,LLD);</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
198	<code>.min(LD,d(_,NewG));</code>	Seleciona o menor termo da lista ordenável de distâncias, obtendo o próximo alvo.
199	<code>.print("Next gold is ",NewG);</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
200	<code>!!handle(NewG).</code>	Inicia o objetivo indicado numa intenção separada, permitindo sair da preparação atômica da coleta.
201	<code>-!choose_gold <- --free.</code>	Trata a falha do objetivo de escolher outro ouro conhecido ou voltar à exploração.
203	<code>+!calc_gold_distance([],[]).</code>	Oferece um plano para montar a lista de distâncias dos alvos conhecidos.
204	<code>+!calc_gold_distance([gold(GX,GY) R], [d(D,gold(GX,GY)) RD])</code>	Oferece um plano para montar a lista de distâncias dos alvos conhecidos.
205	<code>: pos(IX,IY)</code>	Exige este contexto para selecionar o plano; & combina condições e not consulta a ausência de uma solução na base de crenças.
206	<code><- jia.dist(IX,IY,GX,GY,D);</code>	Calcula a distância entre as coordenadas para comparar os alvos conhecidos.
207	<code>!calc_gold_distance(R,RD).</code>	Solicita o subobjetivo de montar a lista de distâncias dos alvos conhecidos. O ponto e vírgula liga a próxima etapa do corpo.
208	<code>+!calc_gold_distance([_ R],RD)</code>	Oferece um plano para montar a lista de distâncias dos alvos conhecidos.
209	<code><- !calc_gold_distance(R,RD).</code>	Solicita o subobjetivo de montar a lista de distâncias dos alvos conhecidos. O ponto e vírgula liga a próxima etapa do corpo.
214	<code>+end_of_simulation(S,_) : true</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
215	<code><- .drop_all_desires;</code>	Abandona os objetivos ativos ao receber o encerramento da simulação.
216	<code>.abolish(gold(_,_));</code>	Remove as crenças que correspondem ao padrão, inclusive quando possuem anotações de origem.
217	<code>.abolish(picked(_));</code>	Remove as crenças que correspondem ao padrão, inclusive quando possuem anotações de origem.
218	<code>--free;</code>	Substitui a crença indicada por seu novo valor, gerando o evento correspondente à atualização.
219	<code>.print("-- END ",S," --").</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.

src/agt/miner.asl

Explora o mapa e escolhe destinos. Comece pelos planos de free, near e next_step; esta versão não faz a entrega de ouro.

[Abrir arquivo original](#) · 90 linhas · numeração da cópia documentada

Linha	Código	Como ler
3	<pre>{ include("\$jacamoJar/templates/common-cartago.asl") }</pre>	Inclui os planos comuns de integração Jason-CARTaGO fornecidos pelo JaCaMo.
11	<pre>last_dir(null). // the last movement I did</pre>	Compõe o plano ou a crença deste trecho de AgentSpeak. Leia em conjunto com o evento acima e a condição que permite adotá-lo.
12	<pre>free.</pre>	Crença inicial de disponibilidade para explorar ou assumir uma coleta.
28	<pre>+free : gsize(_,W,H) & jia.random(RX,W-1) & jia.random(RY,H-1)</pre>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
29	<pre><- !go_near(RX,RY).</pre>	Solicita o subobjetivo de iniciar a exploração em direção a um destino escolhido. O ponto e vírgula liga a próxima etapa do corpo.
30	<pre>+free // gsize is unknown yet</pre>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
31	<pre><- .wait(100); ++free.</pre>	Espera o intervalo indicado antes de repetir a observação. A espera evita uma repetição imediata sem novas percepções.
37	<pre>+near(X,Y) : free <- ++free.</pre>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
51	<pre>+!go_near(X,Y) : free</pre>	Oferece um plano para iniciar a exploração em direção a um destino escolhido. O trecho após os dois-pontos é o contexto de seleção.
52	<pre><- -near(_,_);</pre>	Retira a crença indicada, porque ela deixou de representar a situação que o agente quer manter.
53	<pre>-last_dir(_);</pre>	Retira a crença indicada, porque ela deixou de representar a situação que o agente quer manter.
54	<pre>!near(X,Y).</pre>	Solicita o subobjetivo de chegar à vizinhança do destino ou tratar uma rota não encontrada. O ponto e vírgula liga a próxima etapa do corpo.
57	<pre>+!near(X,Y) : (pos(AgX,AgY) & jia.neighbour(AgX,AgY,X,Y)) last_dir(skip) // I am near to some location if I am near it or the last action was skip (meaning that there are no paths to there)</pre>	Oferece um plano para chegar à vizinhança do destino ou tratar uma rota não encontrada. O trecho após os dois-pontos é o contexto de seleção.
58	<pre><- +near(X,Y).</pre>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
59	<pre>+!near(X,Y) : not near(X,Y)</pre>	Oferece um plano para chegar à vizinhança do destino ou tratar uma rota não encontrada. O trecho após os dois-pontos é o contexto de seleção.
60	<pre><- !next_step(X,Y);</pre>	Solicita o subobjetivo de escolher e executar um único movimento. O ponto e vírgula liga a próxima etapa do corpo.

Linha	Código	Como ler
61	<code>!near(X,Y).</code>	Solicita o subobjetivo de chegar à vizinhança do destino ou tratar uma rota não encontrada. O ponto e vírgula liga a próxima etapa do corpo.
62	<code>+!near(X,Y) : true</code>	Oferece um plano para chegar à vizinhança do destino ou tratar uma rota não encontrada. O trecho após os dois-pontos é o contexto de seleção.
63	<code><- !near(X,Y).</code>	Solicita o subobjetivo de chegar à vizinhança do destino ou tratar uma rota não encontrada. O ponto e vírgula liga a próxima etapa do corpo.
71	<code>+!next_step(X,Y) : pos(AgX,AgY) // I already know my position</code>	Oferece um plano para escolher e executar um único movimento. O trecho após os dois-pontos é o contexto de seleção.
72	<code><- jia.get_direction(AgX, AgY, X, Y, D);</code>	Pede à ação interna uma direção para o alvo. A implementação usa busca A* e ainda consulta o mapa global.
73	<code>--last_dir(D);</code>	Substitui a crença indicada por seu novo valor, gerando o evento correspondente à atualização.
74	<code>D.</code>	Executa como ação a direção que foi unificada com a variável D.
75	<code>+!next_step(X,Y) : not pos(_,_) // I still do not know my position</code>	Oferece um plano para escolher e executar um único movimento. O trecho após os dois-pontos é o contexto de seleção.
76	<code><- !next_step(X,Y).</code>	Solicita o subobjetivo de escolher e executar um único movimento. O ponto e vírgula liga a próxima etapa do corpo.
77	<code>-!next_step(X,Y) : true // failure handling -> start again!</code>	Trata a falha do objetivo de escolher e executar um único movimento. O trecho após os dois-pontos é o contexto de seleção.
78	<code><- --last_dir(null);</code>	Substitui a crença indicada por seu novo valor, gerando o evento correspondente à atualização.
79	<code>!next_step(X,Y).</code>	Solicita o subobjetivo de escolher e executar um único movimento. O ponto e vírgula liga a próxima etapa do corpo.
84	<code>+end_of_simulation(S,_) : true</code>	Adiciona uma crença ou define reação à sua chegada. O contexto e o corpo, quando presentes, determinam o comportamento.
85	<code><- .drop_all_desires;</code>	Abandona os objetivos ativos ao receber o encerramento da simulação.
86	<code>.abolish(gold(_,_));</code>	Remove as crenças que correspondem ao padrão, inclusive quando possuem anotações de origem.
87	<code>.abolish(picked(_));</code>	Remove as crenças que correspondem ao padrão, inclusive quando possuem anotações de origem.
88	<code>--free;</code>	Substitui a crença indicada por seu novo valor, gerando o evento correspondente à atualização.
89	<code>.print("-- END ",S," --").</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.

src/test/agt/verificacao.asl

Percorre uma coleta conhecida. Espera perceber cada efeito antes de avançar e pede uma conferência independente ao final.

Linha	Código	Como ler
2	<code>{ include("\$jacamoJar/templates/common-cartago.asl") }</code>	Inclui os planos comuns de integração Jason–CARTaGO fornecidos pelo JaCaMo.
3	<code>!verificar.</code>	Solicita o subobjetivo de percorrer e conferir a entrega do teste. O ponto e vírgula liga a próxima etapa do corpo.
5	<code>+!verificar</code>	Oferece um plano para percorrer e conferir a entrega do teste.
6	<code><- !posicao(1,0);</code>	Solicita o subobjetivo de esperar que a posição pretendida apareça na percepção. O ponto e vírgula liga a próxima etapa do corpo.
7	<code>?cell(1,1,gold);</code>	Consulta a base de crenças e exige que a condição indicada seja satisfeita para continuar.
8	<code>.print("GOLD_MINERS PERCEPTION gold=1,1");</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
9	<code>down;</code>	Solicita a operação down ao artefato em foco. Os efeitos serão observados pelo agente.
10	<code>!posicao(1,1);</code>	Solicita o subobjetivo de esperar que a posição pretendida apareça na percepção. O ponto e vírgula liga a próxima etapa do corpo.
11	<code>pick;</code>	Solicita a operação pick ao artefato em foco. Os efeitos serão observados pelo agente.
12	<code>!com_carga;</code>	Solicita o subobjetivo de esperar a confirmação da carga. O ponto e vírgula liga a próxima etapa do corpo.
13	<code>?not cell(1,1,gold);</code>	Consulta a base de crenças e exige que a condição indicada seja satisfeita para continuar.
14	<code>.print("GOLD_MINERS PICK carrying=true");</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
15	<code>up;</code>	Solicita a operação up ao artefato em foco. Os efeitos serão observados pelo agente.
16	<code>!posicao(1,0);</code>	Solicita o subobjetivo de esperar que a posição pretendida apareça na percepção. O ponto e vírgula liga a próxima etapa do corpo.
17	<code>left;</code>	Solicita a operação left ao artefato em foco. Os efeitos serão observados pelo agente.
18	<code>!posicao(0,0);</code>	Solicita o subobjetivo de esperar que a posição pretendida apareça na percepção. O ponto e vírgula liga a próxima etapa do corpo.
19	<code>drop;</code>	Solicita a operação drop ao artefato em foco. Os efeitos serão observados pelo agente.
20	<code>!sem_carga;</code>	Solicita o subobjetivo de esperar a confirmação de carga vazia. O ponto e vírgula liga a próxima etapa do corpo.
21	<code>validateDelivery;</code>	Solicita a operação validateDelivery ao artefato em foco. Os efeitos serão observados pelo agente.
22	<code>.print("GOLD_MINERS END status=ok");</code>	Registra uma mensagem para acompanhar a execução. Leia-a junto às percepções e ao estado do mundo para entender o que foi confirmado.
23	<code>.stopMAS.</code>	Encerra o sistema do teste após registrar o resultado.

Linha	Código	Como ler
25	<code>+!posicao(X,Y) : pos(X,Y).</code>	Oferece um plano para esperar que a posição pretendida apareça na percepção. O trecho após os dois-pontos é o contexto de seleção.
26	<code>+!posicao(X,Y) : not pos(X,Y) <- .wait(20); !posicao(X,Y).</code>	Oferece um plano para esperar que a posição pretendida apareça na percepção. O trecho após os dois-pontos é o contexto de seleção.
27	<code>+!com_carga : carrying_gold.</code>	Oferece um plano para esperar a confirmação da carga. O trecho após os dois-pontos é o contexto de seleção.
28	<code>+!com_carga : not carrying_gold <- .wait(20); !com_carga.</code>	Oferece um plano para esperar a confirmação da carga. O trecho após os dois-pontos é o contexto de seleção.
29	<code>+!sem_carga : not carrying_gold.</code>	Oferece um plano para esperar a confirmação de carga vazia. O trecho após os dois-pontos é o contexto de seleção.
30	<code>+!sem_carga : carrying_gold <- .wait(20); !sem_carga.</code>	Oferece um plano para esperar a confirmação de carga vazia. O trecho após os dois-pontos é o contexto de seleção.
31	<code>-!verificar <- .print("GOLD_MINERS FAIL agent_verification"); .stopMAS.</code>	Trata a falha do objetivo de percorrer e conferir a entrega do teste.

src/agt/jia/dist.java

Converte coordenadas do AgentSpeak e devolve a distância calculada por Location. Distância geométrica não demonstra que exista caminho livre.

[Abrir arquivo original](#) · 27 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>package jia;</code>	Agrupar a classe no pacote jia, usado para localizá-la no projeto.
3	<code>import jason.asSemantics.DefaultInternalAction;</code>	Fornecer a base Java para criar uma ação interna chamável em AgentSpeak.
4	<code>import jason.asSemantics.TransitionSystem;</code>	Dá acesso ao contexto de execução do agente Jason.
5	<code>import jason.asSemantics.Unifier;</code>	Relaciona os termos e variáveis AgentSpeak com os resultados produzidos em Java.
6	<code>import jason.asSyntax.NumberTerm;</code>	Representa um termo numérico recebido ou avaliado pelo agente.
7	<code>import jason.asSyntax.NumberTermImpl;</code>	Cria um termo numérico para devolver um resultado ao AgentSpeak.
8	<code>import jason.asSyntax.Term;</code>	Representa os argumentos simbólicos das ações internas.
9	<code>import jason.environment.grid.Location;</code>	Representa coordenadas e oferece operações de posição e distância.
11	<code>public class dist extends DefaultInternalAction {</code>	Define uma classe e, quando indicado por extends ou implements, relaciona seu comportamento à base ou às interfaces utilizadas.
13	<code>@Override</code>	Indica que o método redefine um comportamento previsto na classe ou interface de origem.
14	<code>public Object execute(TransitionSystem ts, Unifier un, Term[] terms) throws Exception {</code>	Inicia o método para executar a ação interna dist e devolver seu resultado ao Jason.

Linha	Código	Como ler
15	<code>try {</code>	Abre um trecho cujo erro será tratado pelos blocos catch seguintes.
16	<code>int iagx = (int) ((NumberTerm) terms[0]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
17	<code>int iagy = (int) ((NumberTerm) terms[1]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
18	<code>int itox = (int) ((NumberTerm) terms[2]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
19	<code>int itoy = (int) ((NumberTerm) terms[3]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
20	<code>int dist = new Location(iagx, iagy).distance(new Location(itox, itoy));</code>	Define ou atualiza int dist com a expressão à direita. Essa informação participa da rotina de executar a ação interna dist e devolver seu resultado ao Jason.
21	<code>return un.unifies(terms[4], new NumberTermImpl(dist));</code>	Entrega este resultado a quem chamou a rotina de executar a ação interna dist e devolver seu resultado ao Jason.
22	<code>} catch (Throwable e) {</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
23	<code>e.printStackTrace();</code>	Executa a chamada indicada como parte da rotina de executar a ação interna dist e devolver seu resultado ao Jason. Seus argumentos indicam os dados usados nesta etapa.
24	<code>return false;</code>	Devolve falso para indicar que a condição ou tentativa não foi satisfeita.
25	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
26	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
27	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

src/agt/jia/get_direction.java

Usa A* para escolher um primeiro movimento. Consulta o modelo global; esse acesso deverá mudar para uma competição com informação local.

[Abrir arquivo original](#) · 134 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>package jia;</code>	Agrupa a classe no pacote jia, usado para localizá-la no projeto.
3	<code>import jason.asSemantics.DefaultInternalAction;</code>	Fornece a base Java para criar uma ação interna chamável em AgentSpeak.
4	<code>import jason.asSemantics.TransitionSystem;</code>	Dá acesso ao contexto de execução do agente Jason.
5	<code>import jason.asSemantics.Unifier;</code>	Relaciona os termos e variáveis AgentSpeak com os resultados produzidos em Java.
6	<code>import jason.asSyntax.Atom;</code>	Representa um símbolo AgentSpeak, como uma direção de movimento.
7	<code>import jason.asSyntax.NumberTerm;</code>	Representa um termo numérico recebido ou avaliado pelo agente.

Linha	Código	Como ler
8	<code>import jason.asSyntax.Term;</code>	Representa os argumentos simbólicos das ações internas.
9	<code>import jason.environment.grid.GridWorldModel;</code>	Fornecer a representação básica de um mundo em grade, posições e ocupação.
10	<code>import jason.environment.grid.Location;</code>	Representa coordenadas e oferece operações de posição e distância.
12	<code>import java.util.ArrayList;</code>	Cria uma lista redimensionável, usada para reunir sucessores.
13	<code>import java.util.List;</code>	Define o tipo de lista usado para os estados candidatos.
15	<code>import mining.WorldModel;</code>	Traz o modelo do domínio. O uso global pela política é uma limitação conhecida do tutorial.
16	<code>import busca.AEstrela;</code>	Traz a implementação do algoritmo A* usada no cálculo de direção.
17	<code>import busca.Busca;</code>	Define a interface de busca usada para executar o algoritmo selecionado.
18	<code>import busca.Estado;</code>	Representa um estado do problema de busca.
19	<code>import busca.Heuristica;</code>	Define como fornecer uma estimativa de custo restante à busca.
20	<code>import busca.Nodo;</code>	Representa um nó e seu vínculo com o pai no caminho encontrado.
22	<code>public class get_direction extends DefaultInternalAction {</code>	Define uma classe e, quando indicado por extends ou implements, relaciona seu comportamento à base ou às interfaces utilizadas.
24	<code>@Override</code>	Indica que o método redefine um comportamento previsto na classe ou interface de origem.
25	<code>public Object execute(TransitionSystem ts, Unifier un, Term[] terms) throws Exception {</code>	Inicia o método para executar a ação interna get_direction e devolver seu resultado ao Jason.
26	<code>try {</code>	Abre um trecho cujo erro será tratado pelos blocos catch seguintes.
27	<code>String sAction = "skip";</code>	Define ou atualiza String sAction com a expressão à direita. Essa informação participa da rotina de executar a ação interna get_direction e devolver seu resultado ao Jason.
29	<code>WorldModel model = WorldModel.get();</code>	Obtém o mundo compartilhado. Numa política competitiva, esse acesso global precisa ser substituído por conhecimento permitido.
31	<code>int iagx = (int) ((NumberTerm)terms[0]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
32	<code>int iagy = (int) ((NumberTerm)terms[1]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
33	<code>int itox = (int) ((NumberTerm)terms[2]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
34	<code>int itoy = (int) ((NumberTerm)terms[3]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
36	<code>if (model.inGrid(itox,itoy)) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é executar a ação interna get_direction e devolver seu resultado ao Jason.
37	<code>Busca searchAlg = new AEstrela();</code>	Escolhe A* como algoritmo de busca para calcular a direção do próximo movimento.

Linha	Código	Como ler
39	<code>Location lini = new Location(iagx, iagy);</code>	Define ou atualiza Location lini com a expressão à direita. Essa informação participa da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason.
42	<code>while (!model.isFreeOfObstacle(itox,itoy) && itox > 0) itox--;</code>	Repete o trecho enquanto a condição continuar verdadeira. Observe o que muda a cada repetição para entender quando ela termina.
43	<code>while (!model.isFreeOfObstacle(itox,itoy) && itox < model.getWidth()) itox++;</code>	Repete o trecho enquanto a condição continuar verdadeira. Observe o que muda a cada repetição para entender quando ela termina.
45	<code>Nodo solution = searchAlg.busca(new GridState(lini, lini,new Location(itox, itoy), model, "initial"));</code>	Executa a busca a partir da posição atual até o destino representado pelo estado inicial.
46	<code>if (solution != null) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason.
48	<code>Nodo root = solution;</code>	Define ou atualiza Nodo root com a expressão à direita. Essa informação participa da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason.
49	<code>Estado prev1 = null;</code>	Define ou atualiza Estado prev1 com a expressão à direita. Essa informação participa da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason.
50	<code>Estado prev2 = null;</code>	Define ou atualiza Estado prev2 com a expressão à direita. Essa informação participa da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason.
51	<code>while (root != null) {</code>	Repete o trecho enquanto a condição continuar verdadeira. Observe o que muda a cada repetição para entender quando ela termina.
52	<code>prev2 = prev1;</code>	Define ou atualiza prev2 com a expressão à direita. Essa informação participa da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason.
53	<code>prev1 = root.getEstado();</code>	Obtém o estado deste nó do caminho encontrado pela busca.
54	<code>root = root.getPai();</code>	Volta ao estado pai para percorrer a solução da busca em direção à sua origem.
55	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
56	<code>if (prev2 != null) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason.
57	<code>sAction = ((GridState)prev2).op;</code>	Define ou atualiza sAction com a expressão à direita. Essa informação participa da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason.
58	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
59	<code>} else {</code>	Trata a alternativa em que a condição anterior não foi satisfeita.
61	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
62	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

Linha	Código	Como ler
63	<code>return un.unifies(terms[4], new Atom(sAction));</code>	Entrega este resultado a quem chamou a rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason.
64	<code>} catch (Throwable e) {</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
65	<code>e.printStackTrace();</code>	Executa a chamada indicada como parte da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason. Seus argumentos indicam os dados usados nesta etapa.
66	<code>return false;</code>	Devolve falso para indicar que a condição ou tentativa não foi satisfeita.
67	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
68	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
69	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
71	<code>class GridState implements Estado, Heuristica {</code>	Define uma classe e, quando indicado por <code>extends</code> ou <code>implements</code> , relaciona seu comportamento à base ou às interfaces utilizadas.
74	<code>Location pos; // current location</code>	Continuação da expressão ou estrutura iniciada acima. Leia-a junto à linha anterior para acompanhar a operação completa.
75	<code>Location from,to;</code>	Executa a chamada indicada como parte da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason. Seus argumentos indicam os dados usados nesta etapa.
76	<code>String op;</code>	Executa a chamada indicada como parte da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason. Seus argumentos indicam os dados usados nesta etapa.
77	<code>GridWorldModel model;</code>	Executa a chamada indicada como parte da rotina de executar a ação interna <code>get_direction</code> e devolver seu resultado ao Jason. Seus argumentos indicam os dados usados nesta etapa.
79	<code>public GridState(Location l, Location from, Location to, GridWorldModel model, String op) {</code>	Inicia o método para <code>GridState</code> .
80	<code>this.pos = l;</code>	Define ou atualiza <code>this.pos</code> com a expressão à direita. Essa informação participa da rotina de <code>GridState</code> .
81	<code>this.from = from;</code>	Define ou atualiza <code>this.from</code> com a expressão à direita. Essa informação participa da rotina de <code>GridState</code> .
82	<code>this.to = to;</code>	Define ou atualiza <code>this.to</code> com a expressão à direita. Essa informação participa da rotina de <code>GridState</code> .
83	<code>this.model = model;</code>	Define ou atualiza <code>this.model</code> com a expressão à direita. Essa informação participa da rotina de <code>GridState</code> .
84	<code>this.op = op;</code>	Define ou atualiza <code>this.op</code> com a expressão à direita. Essa informação participa da rotina de <code>GridState</code> .
85	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
87	<code>public int custo() {</code>	Inicia o método para informar o custo de uma transição na busca.
88	<code>return 1;</code>	Atribui custo unitário ao estado/transição usado pela busca do exemplo.
89	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

Linha	Código	Como ler
91	<code>public boolean ehMeta() {</code>	Inicia o método para verificar se a posição do estado coincide com o destino.
92	<code>return pos.equals(to);</code>	Reconhece a meta quando a posição coincide com o destino.
93	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
95	<code>public String getDescricao() {</code>	Inicia o método para fornecer a descrição textual do problema de busca.
96	<code>return "Grid search";</code>	Entrega este resultado a quem chamou a rotina de fornecer a descrição textual do problema de busca.
97	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
99	<code>public int h() {</code>	Inicia o método para estimar a distância restante até o destino.
100	<code>return pos.distance(to);</code>	Usa a distância de Location como estimativa. Ela não incorpora, por si só, todos os desvios por obstáculos.
101	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
103	<code>public List<Estado> sucessores() {</code>	Inicia o método para construir estados candidatos para os quatro movimentos.
104	<code>List<Estado> s = new ArrayList<Estado>(4);</code>	Define ou atualiza List<Estado> s com a expressão à direita. Essa informação participa da rotina de construir estados candidatos para os quatro movimentos.
106	<code>suc(s,new Location(pos.x-1,pos.y),"left");</code>	Propõe um dos movimentos vizinhos para a busca; a rotina suc decide se o estado pode entrar na lista.
107	<code>suc(s,new Location(pos.x+1,pos.y),"right");</code>	Propõe um dos movimentos vizinhos para a busca; a rotina suc decide se o estado pode entrar na lista.
108	<code>suc(s,new Location(pos.x,pos.y-1),"up");</code>	Propõe um dos movimentos vizinhos para a busca; a rotina suc decide se o estado pode entrar na lista.
109	<code>suc(s,new Location(pos.x,pos.y+1),"down");</code>	Propõe um dos movimentos vizinhos para a busca; a rotina suc decide se o estado pode entrar na lista.
110	<code>return s;</code>	Entrega este resultado a quem chamou a rotina de construir estados candidatos para os quatro movimentos.
111	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
113	<code>private void suc(List<Estado> s, Location newl, String op) {</code>	Inicia o método para adicionar um sucessor quando a regra de ocupação permite.
114	<code>if (model.isFree(newl) (from.distance(newl) > 3 && model.isFreeOfObstacle(newl))) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é adicionar um sucessor quando a regra de ocupação permite.
115	<code>s.add(new GridState(newl,from,to,model,op));</code>	Constrói um estado da busca com posição, origem, destino, modelo e ação associada.
116	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
117	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
119	<code>public boolean equals(Object o) {</code>	Inicia o método para comparar estados pela posição representada.

Linha	Código	Como ler
120	<code>try {</code>	Abre um trecho cujo erro será tratado pelos blocos catch seguintes.
121	<code>GridState m = (GridState)o;</code>	Define ou atualiza GridState m com a expressão à direita. Essa informação participa da rotina de comparar estados pela posição representada.
122	<code>return pos.equals(m.pos);</code>	Entrega este resultado a quem chamou a rotina de comparar estados pela posição representada.
123	<code>} catch (Exception e) {}</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
124	<code>return false;</code>	Devolve falso para indicar que a condição ou tentativa não foi satisfeita.
125	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
127	<code>public int hashCode() {</code>	Inicia o método para produzir o código de dispersão compatível com a posição.
128	<code>return pos.hashCode();</code>	Usa a posição para organizar estados em estruturas de busca que dependem de hash.
129	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
131	<code>public String toString() {</code>	Inicia o método para produzir uma representação textual do objeto.
132	<code>return "(" + pos + "-" + op + ")";</code>	Entrega este resultado a quem chamou a rotina de produzir uma representação textual do objeto.
133	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
134	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

src/agt/jia/neighbour.java

Verifica proximidade usando a classe Location e devolve verdadeiro ou falso para o contexto do plano.

[Abrir arquivo original](#) · 26 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>package jia;</code>	Agrupa a classe no pacote jia, usado para localizá-la no projeto.
3	<code>import jason.asSemantics.DefaultInternalAction;</code>	Fornece a base Java para criar uma ação interna chamável em AgentSpeak.
4	<code>import jason.asSemantics.TransitionSystem;</code>	Dá acesso ao contexto de execução do agente Jason.
5	<code>import jason.asSemantics.Unifier;</code>	Relaciona os termos e variáveis AgentSpeak com os resultados produzidos em Java.
6	<code>import jason.asSyntax.NumberTerm;</code>	Representa um termo numérico recebido ou avaliado pelo agente.
7	<code>import jason.asSyntax.Term;</code>	Representa os argumentos simbólicos das ações internas.
8	<code>import jason.environment.grid.Location;</code>	Representa coordenadas e oferece operações de posição e distância.

Linha	Código	Como ler
10	<code>public class neighbour extends DefaultInternalAction {</code>	Define uma classe e, quando indicado por <code>extends</code> ou <code>implements</code> , relaciona seu comportamento à base ou às interfaces utilizadas.
12	<code>@Override</code>	Indica que o método redefine um comportamento previsto na classe ou interface de origem.
13	<code>public Object execute(TransitionSystem ts, Unifier un, Term[] terms) throws Exception {</code>	Inicia o método para executar a ação interna <code>neighbour</code> e devolver seu resultado ao Jason.
14	<code>try {</code>	Abre um trecho cujo erro será tratado pelos blocos <code>catch</code> seguintes.
15	<code>int iagx = (int) ((NumberTerm)terms[0]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
16	<code>int iagy = (int) ((NumberTerm)terms[1]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
17	<code>int itox = (int) ((NumberTerm)terms[2]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
18	<code>int itoy = (int) ((NumberTerm)terms[3]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
20	<code>return new Location(iagx, iagy).isNeighbour(new Location(itox, itoy));</code>	Entrega este resultado a quem chamou a rotina de executar a ação interna <code>neighbour</code> e devolver seu resultado ao Jason.
21	<code>} catch (Throwable e) {</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
22	<code>e.printStackTrace();</code>	Executa a chamada indicada como parte da rotina de executar a ação interna <code>neighbour</code> e devolver seu resultado ao Jason. Seus argumentos indicam os dados usados nesta etapa.
23	<code>return false;</code>	Devolve falso para indicar que a condição ou tentativa não foi satisfeita.
24	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
25	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
26	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

src/agt/jia/random.java

Gera números aleatórios como ação interna Jason. O limite de `nextInt` é exclusivo; isso deve ser considerado ao passar dimensões do mapa.

[Abrir arquivo original](#) · 55 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>package jia;</code>	Agrupa a classe no pacote <code>jia</code> , usado para localizá-la no projeto.
3	<code>import jason.JsonException;</code>	Permite comunicar erros de uso de uma ação interna ao Jason.
4	<code>import jason.asSemantics.DefaultInternalAction;</code>	Fornece a base Java para criar uma ação interna chamável em <code>AgentSpeak</code> .
5	<code>import jason.asSemantics.TransitionSystem;</code>	Dá acesso ao contexto de execução do agente Jason.

Linha	Código	Como ler
6	<code>import jason.asSemantics.Unifier;</code>	Relaciona os termos e variáveis AgentSpeak com os resultados produzidos em Java.
7	<code>import jason.asSyntax.NumberTermImpl;</code>	Cria um termo numérico para devolver um resultado ao AgentSpeak.
8	<code>import jason.asSyntax.NumberTerm;</code>	Representa um termo numérico recebido ou avaliado pelo agente.
9	<code>import jason.asSyntax.Term;</code>	Representa os argumentos simbólicos das ações internas.
11	<code>import java.util.Random;</code>	Produz os sorteios usados na exploração; esta base ainda não controla a semente pela configuração.
12	<code>import java.util.Iterator;</code>	Oferece resultados sucessivos à consulta que pede números aleatórios.
14	<code>public class random extends DefaultInternalAction {</code>	Define uma classe e, quando indicado por extends ou implements, relaciona seu comportamento à base ou às interfaces utilizadas.
16	<code>private Random random = new Random();</code>	Define ou atualiza private Random random com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
18	<code>@Override</code>	Indica que o método redefine um comportamento previsto na classe ou interface de origem.
19	<code>public Object execute(final TransitionSystem ts, final Unifier un, final Term[] args) throws Exception {</code>	Inicia o método para executar a ação interna random e devolver seu resultado ao Jason.
20	<code>try {</code>	Abre um trecho cujo erro será tratado pelos blocos catch seguintes.
21	<code>if (!args[0].isVar()) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é executar a ação interna random e devolver seu resultado ao Jason.
22	<code>throw new JasonException("The first argument of the internal action 'random' is not a variable.");</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
23	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
24	<code>if (!args[1].isNumeric()) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é executar a ação interna random e devolver seu resultado ao Jason.
25	<code>throw new JasonException("The second argument of the internal action 'random' is not a number.");</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
26	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
27	<code>final int max = (int) ((NumberTerm)args[1]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
29	<code>final int maxIter = args.length < 3 ? Integer.MAX_VALUE : (int) ((NumberTerm)args[2]).solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
31	<code>return new Iterator<Unifier>() {</code>	Entrega este resultado a quem chamou a rotina de executar a ação interna random e devolver seu resultado ao Jason.
32	<code>int i = 0;</code>	Define ou atualiza int i com a expressão à direita. Essa informação participa da rotina de executar a ação interna random e devolver seu resultado ao Jason.

Linha	Código	Como ler
35	<code>public boolean hasNext() {</code>	Inicia o método para verificar se ainda pode ser produzido outro resultado.
36	<code>return i < maxIter && ts.getUserAgArch().isRunning();</code>	Entrega este resultado a quem chamou a rotina de verificar se ainda pode ser produzido outro resultado.
37	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
39	<code>public Unifier next() {</code>	Inicia o método para produzir um novo resultado aleatório com uma cópia do unificador.
40	<code>i++;</code>	Executa a chamada indicada como parte da rotina de produzir um novo resultado aleatório com uma cópia do unificador. Seus argumentos indicam os dados usados nesta etapa.
41	<code>Unifier c = un.clone();</code>	Define ou atualiza Unifier c com a expressão à direita. Essa informação participa da rotina de produzir um novo resultado aleatório com uma cópia do unificador.
42	<code>c.unifies(args[0], new NumberTermImpl(random.nextInt(max)));</code>	Unifica o resultado Java com o argumento AgentSpeak, preenchendo a variável de saída quando houver correspondência.
43	<code>return c;</code>	Entrega este resultado a quem chamou a rotina de produzir um novo resultado aleatório com uma cópia do unificador.
44	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
46	<code>public void remove() {}</code>	Executa a chamada indicada como parte da rotina de produzir um novo resultado aleatório com uma cópia do unificador. Seus argumentos indicam os dados usados nesta etapa.
47	<code>};</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
49	<code>} catch (ArrayIndexOutOfBoundsException e) {</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
50	<code>throw new JSONException("The internal action 'random' has not received the required argument.");</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
51	<code>} catch (Exception e) {</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
52	<code>throw new JSONException("Error in internal action 'random': " + e, e);</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
53	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
54	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
55	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

src/agt/jia/random_direction.java

Alternativa antiga não usada pelos mineradores executados. As coordenadas candidatas começam em -1 e não são reinicializadas por tentativa; revisar antes de usar.

[Abrir arquivo original](#) · 58 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>package jia;</code>	Agrupa a classe no pacote jia, usado para localizá-la no projeto.
3	<code>import jason.asSemantics.DefaultInternalAction;</code>	Fornecer a base Java para criar uma ação interna chamável em AgentSpeak.
4	<code>import jason.asSemantics.TransitionSystem;</code>	Dá acesso ao contexto de execução do agente Jason.
5	<code>import jason.asSemantics.Unifier;</code>	Relaciona os termos e variáveis AgentSpeak com os resultados produzidos em Java.
6	<code>import jason.asSyntax.Atom;</code>	Representa um símbolo AgentSpeak, como uma direção de movimento.
7	<code>import jason.asSyntax.NumberTerm;</code>	Representa um termo numérico recebido ou avaliado pelo agente.
8	<code>import jason.asSyntax.Term;</code>	Representa os argumentos simbólicos das ações internas.
10	<code>import java.util.Random;</code>	Produz os sorteios usados na exploração; esta base ainda não controla a semente pela configuração.
12	<code>import mining.WorldModel;</code>	Traz o modelo do domínio. O uso global pela política é uma limitação conhecida do tutorial.
15	<code>public class random_direction extends DefaultInternalAction {</code>	Define uma classe e, quando indicado por extends ou implements, relaciona seu comportamento à base ou às interfaces utilizadas.
16	<code>Random rnd = new Random();</code>	Define ou atualiza Random rnd com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
18	<code>@Override</code>	Indica que o método redefine um comportamento previsto na classe ou interface de origem.
19	<code>public Object execute(TransitionSystem ts, Unifier un, Term[] terms) throws Exception {</code>	Inicia o método para executar a ação interna random_direction e devolver seu resultado ao Jason.
20	<code>try {</code>	Abre um trecho cujo erro será tratado pelos blocos catch seguintes.
21	<code>String sAction = null;</code>	Define ou atualiza String sAction com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
23	<code>WorldModel model = WorldModel.get();</code>	Obtém o mundo compartilhado. Numa política competitiva, esse acesso global precisa ser substituído por conhecimento permitido.
25	<code>NumberTerm agx = (NumberTerm) terms[0];</code>	Define ou atualiza NumberTerm agx com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
26	<code>NumberTerm agy = (NumberTerm) terms[1];</code>	Define ou atualiza NumberTerm agy com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
28	<code>int iagx = (int) agx.solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
29	<code>int iagy = (int) agy.solve();</code>	Avalia o termo numérico recebido do agente e o converte para o tipo Java usado no cálculo.
30	<code>int itox = -1;</code>	Define ou atualiza int itox com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
31	<code>int itoy = -1;</code>	Define ou atualiza int itoy com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.

Linha	Código	Como ler
32	<code>while (!model.isFree(itox, itoy)) {</code>	Repete o trecho enquanto a condição continuar verdadeira. Observe o que muda a cada repetição para entender quando ela termina.
33	<code>switch (rnd.nextInt(4)) {</code>	Escolhe um ramo de execução de acordo com o valor indicado.
34	<code>case 0:</code>	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
35	<code>itox = iagx - 1;</code>	Define ou atualiza itox com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
36	<code>sAction = "left";</code>	Define ou atualiza sAction com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
37	<code>break;</code>	Encerra este ramo do switch; evita continuar pelo caso seguinte.
38	<code>case 1:</code>	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
39	<code>itox = iagx + 1;</code>	Define ou atualiza itox com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
40	<code>sAction = "right";</code>	Define ou atualiza sAction com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
41	<code>break;</code>	Encerra este ramo do switch; evita continuar pelo caso seguinte.
42	<code>case 2:</code>	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
43	<code>itoy = iagy - 1;</code>	Define ou atualiza itoy com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
44	<code>sAction = "up";</code>	Define ou atualiza sAction com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
45	<code>break;</code>	Encerra este ramo do switch; evita continuar pelo caso seguinte.
46	<code>case 3:</code>	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
47	<code>itoy = iagy + 1;</code>	Define ou atualiza itoy com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
48	<code>sAction = "down";</code>	Define ou atualiza sAction com a expressão à direita. Essa informação participa da rotina de executar a ação interna random_direction e devolver seu resultado ao Jason.
49	<code>break;</code>	Encerra este ramo do switch; evita continuar pelo caso seguinte.
50	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
51	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

Linha	Código	Como ler
52	<code>return un.unifies(terms[2], new Atom(sAction));</code>	Entrega este resultado a quem chamou a rotina de executar a ação interna <code>random_direction</code> e devolver seu resultado ao Jason.
53	<code>} catch (Throwable e) {</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
54	<code>e.printStackTrace();</code>	Executa a chamada indicada como parte da rotina de executar a ação interna <code>random_direction</code> e devolver seu resultado ao Jason. Seus argumentos indicam os dados usados nesta etapa.
55	<code>return false;</code>	Devolve falso para indicar que a condição ou tentativa não foi satisfeita.
56	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
57	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
58	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

src/env/mining/MiningPlanet.java

Liga ações dos agentes ao mundo por CArtaGO. Expõe posição, carga e células percebidas; não é ainda um coordenador de duas equipes.

[Abrir arquivo original](#) · 159 linhas · numeração da cópia documentada

Linha	Código	Como ler
2	<code>package mining;</code>	Agrupa a classe no pacote <code>mining</code> , usado para localizá-la no projeto.
4	<code>import jason.asSyntax.Atom;</code>	Representa um símbolo <code>AgentSpeak</code> , como uma direção de movimento.
5	<code>import jason.asSyntax.Term;</code>	Representa os argumentos simbólicos das ações internas.
6	<code>import jason.environment.grid.Location;</code>	Representa coordenadas e oferece operações de posição e distância.
8	<code>import java.util.logging.Logger;</code>	Permite escrever informações e avisos para acompanhar a execução.
10	<code>import cartago.Artifact;</code>	Fornece a base para construir um artefato CArtaGO.
11	<code>import cartago.OPERATION;</code>	Marca um método como operação do artefato acessível aos agentes.
12	<code>import cartago.ObsProperty;</code>	Representa uma propriedade observável do artefato.
14	<code>public class MiningPlanet extends Artifact {</code>	Define uma classe e, quando indicado por <code>extends</code> ou <code>implements</code> , relaciona seu comportamento à base ou às interfaces utilizadas.
16	<code>private static Logger logger = Logger.getLogger(MiningPlanet.class.getName());</code>	Define ou atualiza <code>private static Logger logger</code> com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
18	<code>static WorldModel model = null;</code>	Define ou atualiza <code>static WorldModel model</code> com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.

Linha	Código	Como ler
19	<code>static WorldView view;</code>	Executa a chamada indicada como parte da rotina de preparação deste arquivo. Seus argumentos indicam os dados usados nesta etapa.
21	<code>static int simId = 5; // type of environment</code>	Define ou atualiza static int simId com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
22	<code>static int sleep = Integer.getInteger("goldminers.delay", 200);</code>	Define ou atualiza static int sleep com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
23	<code>static boolean hasGUI = !Boolean.getBoolean("goldminers.headless");</code>	Define ou atualiza static boolean hasGUI com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
25	<code>int agId = -1;</code>	Define ou atualiza int agId com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
27	<code>public enum Move {</code>	Continuação da expressão ou estrutura iniciada acima. Leia-a junto à linha anterior para acompanhar a operação completa.
28	<code>UP, DOWN, RIGHT, LEFT</code>	Continuação da expressão ou estrutura iniciada acima. Leia-a junto à linha anterior para acompanhar a operação completa.
29	<code>};</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
31	<code>@OPERATION</code>	Expõe uma operação do artefato aos agentes; as condições e o efeito ficam no corpo indicado.
32	<code>public void init(int scenario, int agId) {</code>	Inicia o método para inicializar o artefato com os parâmetros do projeto.
33	<code>this.agId = agId;</code>	Define ou atualiza this.agId com a expressão à direita. Essa informação participa da rotina de inicializar o artefato com os parâmetros do projeto.
34	<code>initWorld(scenario);</code>	Executa a chamada indicada como parte da rotina de inicializar o artefato com os parâmetros do projeto. Seus argumentos indicam os dados usados nesta etapa.
35	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
37	<code>public int getSimId() {</code>	Inicia o método para consultar o identificador do cenário.
38	<code>return simId;</code>	Entrega este resultado a quem chamou a rotina de consultar o identificador do cenário.
39	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
41	<code>public void setSleep(int s) {</code>	Inicia o método para alterar a espera artificial entre operações.
42	<code>sleep = s;</code>	Define ou atualiza sleep com a expressão à direita. Essa informação participa da rotina de alterar a espera artificial entre operações.
43	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
45	<code>@OPERATION void up() throws Exception { move(Move.UP); }</code>	Expõe uma operação do artefato aos agentes; as condições e o efeito ficam no corpo indicado.
46	<code>@OPERATION void down() throws Exception { move(Move.DOWN); }</code>	Expõe uma operação do artefato aos agentes; as condições e o efeito ficam no corpo indicado.
47	<code>@OPERATION void right() throws Exception { move(Move.RIGHT); }</code>	Expõe uma operação do artefato aos agentes; as condições e o efeito ficam no corpo indicado.

Linha	Código	Como ler
48	<code>@OPERATION void left() throws Exception { move(Move.LEFT); }</code>	Expõe uma operação do artefato aos agentes; as condições e o efeito ficam no corpo indicado.
49	<code>void move(Move m) throws Exception {</code>	Inicia o método para tentar mudar a posição conforme a direção e a ocupação da célula.
50	<code>if (sleep > 0) await_time(sleep);</code>	Aplica a espera artificial configurada entre ações; isso não constitui um protocolo de turnos entre equipes.
51	<code>model.move(m, agId);</code>	Pede o movimento ao modelo. A posição percebida depois informa se houve deslocamento.
52	<code>updateAgPercept();</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
53	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
55	<code>@OPERATION void pick() throws Exception {</code>	Expõe uma operação do artefato aos agentes; as condições e o efeito ficam no corpo indicado.
56	<code>if (sleep > 0) await_time(sleep);</code>	Aplica a espera artificial configurada entre ações; isso não constitui um protocolo de turnos entre equipes.
57	<code>model.pick(agId);</code>	Pede ao modelo a coleta para este agente. A atualização posterior da percepção informa o efeito.
58	<code>updateAgPercept();</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
59	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
60	<code>@OPERATION void drop() throws Exception {</code>	Expõe uma operação do artefato aos agentes; as condições e o efeito ficam no corpo indicado.
61	<code>if (sleep > 0) await_time(sleep);</code>	Aplica a espera artificial configurada entre ações; isso não constitui um protocolo de turnos entre equipes.
62	<code>model.drop(agId);</code>	Pede ao modelo a devolução ou depósito da carga, conforme a posição.
63	<code>if (view != null) view.udpateCollectedGolds();</code>	Só atualiza a janela se ela foi criada; isso permite executar o laboratório sem interface gráfica.
64	<code>updateAgPercept();</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
65	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
66	<code>@OPERATION void skip() {</code>	Expõe uma operação do artefato aos agentes; as condições e o efeito ficam no corpo indicado.
67	<code>if (sleep > 0) await_time(sleep);</code>	Aplica a espera artificial configurada entre ações; isso não constitui um protocolo de turnos entre equipes.
68	<code>updateAgPercept();</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
69	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
71	<code>public synchronized void initWorld(int w) {</code>	Inicia o método para escolher o cenário e preparar as propriedades percebidas.
72	<code>simId = w;</code>	Define ou atualiza simId com a expressão à direita. Essa informação participa da rotina de escolher o cenário e preparar as propriedades percebidas.

Linha	Código	Como ler
73	try {	Abre um trecho cujo erro será tratado pelos blocos catch seguintes.
74	if (model == null) {	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é escolher o cenário e preparar as propriedades percebidas.
75	switch (w) {	Escolhe um ramo de execução de acordo com o valor indicado.
76	case 1: model = WorldModel.world1(); break;	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
77	case 2: model = WorldModel.world2(); break;	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
78	case 3: model = WorldModel.world3(); break;	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
79	case 4: model = WorldModel.world4(); break;	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
80	case 5: model = WorldModel.world5(); break;	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
81	case 6: model = WorldModel.world6(); break;	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
82	default:	Trata valores que não coincidiram com os casos anteriores.
83	logger.info("Invalid index!");	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
84	return;	Executa a chamada indicada como parte da rotina de escolher o cenário e preparar as propriedades percebidas. Seus argumentos indicam os dados usados nesta etapa.
85	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
86	if (hasGUI) {	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é escolher o cenário e preparar as propriedades percebidas.
87	view = new WorldView(model);	Define ou atualiza view com a expressão à direita. Essa informação participa da rotina de escolher o cenário e preparar as propriedades percebidas.
88	view.setEnv(this);	Executa a chamada indicada como parte da rotina de escolher o cenário e preparar as propriedades percebidas. Seus argumentos indicam os dados usados nesta etapa.
89	if (view != null) view.udpateCollectedGolds();	Só atualiza a janela se ela foi criada; isso permite executar o laboratório sem interface gráfica.
90	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
91	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
92	defineObsProperty("gsize", simId, model.getWidth(), model.getHeight());	Cria uma propriedade observável do artefato. Os agentes em foco recebem a informação correspondente pela integração com Jason.
93	defineObsProperty("depot", simId, model.getDepot().x, model.getDepot().y);	Cria uma propriedade observável do artefato. Os agentes em foco recebem a informação correspondente pela integração com Jason.

Linha	Código	Como ler
94	<code>defineObsProperty("pos", -1, -1);</code>	Cria uma propriedade observável do artefato. Os agentes em foco recebem a informação correspondente pela integração com Jason.
95	<code>updateAgPercept();</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
97	<code>} catch (Exception e) {</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
98	<code>logger.warning("Error creating world "+e);</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
99	<code>e.printStackTrace();</code>	Executa a chamada indicada como parte da rotina de escolher o cenário e preparar as propriedades percebidas. Seus argumentos indicam os dados usados nesta etapa.
100	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
101	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
103	<code>public void endSimulation() {</code>	Inicia o método para sinalizar o fim e liberar o modelo.
104	<code>defineObsProperty("end_of_simulation", simId, 0);</code>	Cria uma propriedade observável do artefato. Os agentes em foco recebem a informação correspondente pela integração com Jason.
106	<code>if (view != null) view.setVisible(false);</code>	Só atualiza a janela se ela foi criada; isso permite executar o laboratório sem interface gráfica.
107	<code>WorldModel.destroy();</code>	Executa a chamada indicada como parte da rotina de sinalizar o fim e liberar o modelo. Seus argumentos indicam os dados usados nesta etapa.
108	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
110	<code>private void updateAgPercept() {</code>	Inicia o método para atualizar as informações observáveis do minerador.
112	<code>Location l = model.getAgPos(agId);</code>	Consulta a posição do agente no modelo do mundo.
113	<code>ObsProperty p = getObsProperty("pos");</code>	Obtém a propriedade observável existente para atualizá-la ou consultá-la.
114	<code>p.updateValue(0, l.x);</code>	Atualiza um argumento da propriedade observável; o primeiro número identifica a posição desse argumento.
115	<code>p.updateValue(1, l.y);</code>	Atualiza um argumento da propriedade observável; o primeiro número identifica a posição desse argumento.
117	<code>if (model.isCarryingGold(agId)) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é atualizar as informações observáveis do minerador.
118	<code>if (!hasObsProperty("carrying_gold"))</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é atualizar as informações observáveis do minerador.
119	<code>defineObsProperty("carrying_gold");</code>	Cria uma propriedade observável do artefato. Os agentes em foco recebem a informação correspondente pela integração com Jason.
120	<code>} else try {</code>	Trata a alternativa em que a condição anterior não foi satisfeita.
121	<code>removeObsProperty("carrying_gold");</code>	Retira uma propriedade observável que deixou de representar a percepção atual.

Linha	Código	Como ler
122	<code>} catch (IllegalArgumentException e) {}</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
125	<code>while (hasObsProperty("cell")) removeObsProperty("cell");</code>	Retira uma propriedade observável que deixou de representar a percepção atual.
128	<code>updateAgPercept(1.x - 1, 1.y - 1);</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
129	<code>updateAgPercept(1.x - 1, 1.y);</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
130	<code>updateAgPercept(1.x - 1, 1.y + 1);</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
131	<code>updateAgPercept(1.x, 1.y - 1);</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
132	<code>updateAgPercept(1.x, 1.y);</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
133	<code>updateAgPercept(1.x, 1.y + 1);</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
134	<code>updateAgPercept(1.x + 1, 1.y - 1);</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
135	<code>updateAgPercept(1.x + 1, 1.y);</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
136	<code>updateAgPercept(1.x + 1, 1.y + 1);</code>	Atualiza a percepção do agente ou de uma célula da sua vizinhança após observar o mundo.
139	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
141	<code>private static Term gold = new Atom("gold");</code>	Define ou atualiza private static Term gold com a expressão à direita. Essa informação participa da rotina de atualizar as informações observáveis do minerador.
142	<code>private static Term obstacle = new Atom("obstacle");</code>	Define ou atualiza private static Term obstacle com a expressão à direita. Essa informação participa da rotina de atualizar as informações observáveis do minerador.
144	<code>private void updateAgPercept(int x, int y) {</code>	Inicia o método para atualizar as informações observáveis do minerador.
145	<code>if (model == null !model.inGrid(x,y)) return;</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é atualizar as informações observáveis do minerador.
147	<code>if (model.hasObject(WorldModel.OBSTACLE, x, y)) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é atualizar as informações observáveis do minerador.
148	<code>defineObsProperty("cell", x, y, obstacle);</code>	Cria uma propriedade observável do artefato. Os agentes em foco recebem a informação correspondente pela integração com Jason.
149	<code>} else if (model.hasObject(WorldModel.GOLD, x, y)) {</code>	Trata a alternativa em que a condição anterior não foi satisfeita.
150	<code>defineObsProperty("cell", x, y, gold);</code>	Cria uma propriedade observável do artefato. Os agentes em foco recebem a informação correspondente pela integração com Jason.
151	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

Linha	Código	Como ler
157	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
159	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

src/env/mining/WorldModel.java

Guarda o estado do mundo e aplica movimento, coleta e depósito. A segunda metade constrói seis mapas por coordenadas; cada posição faz parte da instância.

[Abrir arquivo original](#) · 507 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	package mining;	Agrupar a classe no pacote mining, usado para localizá-la no projeto.
3	import jason.environment.grid.GridWorldModel;	Fornecer a representação básica de um mundo em grade, posições e ocupação.
4	import jason.environment.grid.Location;	Representar coordenadas e oferece operações de posição e distância.
6	import java.util.HashSet;	Oferece um conjunto que não mantém elementos duplicados.
7	import java.util.Set;	Define a interface de conjuntos usada para registrar os portadores de ouro.
8	import java.util.logging.Logger;	Permite escrever informações e avisos para acompanhar a execução.
10	import mining.MiningPlanet.Move;	Traz os nomes das quatro direções definidos no artefato MiningPlanet.
12	public class WorldModel extends GridWorldModel {	Define uma classe e, quando indicado por extends ou implements, relaciona seu comportamento à base ou às interfaces utilizadas.
14	public static final int GOLD = 16;	Define ou atualiza public static final int GOLD com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
15	public static final int DEPOT = 32;	Define ou atualiza public static final int DEPOT com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
16	public static final int ENEMY = 64;	Define ou atualiza public static final int ENEMY com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
18	Location depot;	Executa a chamada indicada como parte da rotina de preparação deste arquivo. Seus argumentos indicam os dados usados nesta etapa.
19	Set<Integer> agWithGold; // which agent is carrying gold	Continuação da expressão ou estrutura iniciada acima. Leia-a junto à linha anterior para acompanhar a operação completa.
20	int goldsInDepot = 0;	Define ou atualiza int goldsInDepot com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
21	int initialNbGolds = 0;	Define ou atualiza int initialNbGolds com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.

Linha	Código	Como ler
23	<code>private Logger logger = Logger.getLogger("jasonTeamSimLocal.mas2j." + WorldModel.class.getName());</code>	Define ou atualiza private Logger logger com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
25	<code>private String id = "WorldModel";</code>	Define ou atualiza private String id com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
28	<code>protected static WorldModel model = null;</code>	Define ou atualiza protected static WorldModel model com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
30	<code>synchronized public static WorldModel create(int w, int h, int nbAgs) {</code>	Inicia o método para criar a instância compartilhada do modelo quando ela ainda não existe.
31	<code>if (model == null) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é criar a instância compartilhada do modelo quando ela ainda não existe.
32	<code>model = new WorldModel(w, h, nbAgs);</code>	Define ou atualiza model com a expressão à direita. Essa informação participa da rotina de criar a instância compartilhada do modelo quando ela ainda não existe.
33	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
34	<code>return model;</code>	Entrega este resultado a quem chamou a rotina de criar a instância compartilhada do modelo quando ela ainda não existe.
35	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
37	<code>public static WorldModel get() {</code>	Inicia o método para fornecer acesso à instância compartilhada do modelo.
38	<code>return model;</code>	Entrega este resultado a quem chamou a rotina de fornecer acesso à instância compartilhada do modelo.
39	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
41	<code>public static void destroy() {</code>	Inicia o método para liberar a referência estática ao modelo.
42	<code>model = null;</code>	Remove a referência compartilhada. Isso permite criar outro mundo numa inicialização posterior.
43	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
45	<code>private WorldModel(int w, int h, int nbAgs) {</code>	Inicia o método para WorldModel.
46	<code>super(w, h, nbAgs);</code>	Executa a chamada indicada como parte da rotina de WorldModel. Seus argumentos indicam os dados usados nesta etapa.
47	<code>agWithGold = new HashSet<Integer>();</code>	Cria um conjunto sem identificadores repetidos para registrar quem carrega ouro.
48	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
50	<code>public String getId() {</code>	Inicia o método para consultar o nome do cenário.
51	<code>return id;</code>	Entrega este resultado a quem chamou a rotina de consultar o nome do cenário.
52	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

Linha	Código	Como ler
53	<code>public void setId(String id) {</code>	Inicia o método para atribuir um nome ao cenário.
54	<code> this.id = id;</code>	Define ou atualiza <code>this.id</code> com a expressão à direita. Essa informação participa da rotina de atribuir um nome ao cenário.
55	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
56	<code>public String toString() {</code>	Inicia o método para produzir uma representação textual do objeto.
57	<code> return id;</code>	Entrega este resultado a quem chamou a rotina de produzir uma representação textual do objeto.
58	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
60	<code>public Location getDepot() {</code>	Inicia o método para consultar a posição do depósito.
61	<code> return depot;</code>	Entrega este resultado a quem chamou a rotina de consultar a posição do depósito.
62	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
64	<code>public int getGoldsInDepot() {</code>	Inicia o método para consultar o total de entregas contabilizado pelo mundo.
65	<code> return goldsInDepot;</code>	Entrega este resultado a quem chamou a rotina de consultar o total de entregas contabilizado pelo mundo.
66	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
68	<code>public boolean isAllGoldsCollected() {</code>	Inicia o método para comparar o total depositado com a quantidade inicial registrada.
69	<code> return goldsInDepot == initialNbGolds;</code>	Compara o total depositado com a quantidade inicial registrada. A interpretação muda se houver geração dinâmica de recursos.
70	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
72	<code>public void setInitialNbGolds(int i) {</code>	Inicia o método para registrar o total inicial usado nas conferências.
73	<code> initialNbGolds = i;</code>	Define ou atualiza <code>initialNbGolds</code> com a expressão à direita. Essa informação participa da rotina de registrar o total inicial usado nas conferências.
74	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
76	<code>public int getInitialNbGolds() {</code>	Inicia o método para consultar a quantidade inicial registrada para o cenário.
77	<code> return initialNbGolds;</code>	Entrega este resultado a quem chamou a rotina de consultar a quantidade inicial registrada para o cenário.
78	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
80	<code>public boolean isCarryingGold(int ag) {</code>	Inicia o método para verificar se o identificador está no conjunto de portadores.
81	<code> return agWithGold.contains(ag);</code>	Consulta se o agente aparece no conjunto de quem carrega ouro.

Linha	Código	Como ler
82	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
84	public void setDepot(int x, int y) {	Inicia o método para definir a célula que representa o depósito.
85	depot = new Location(x, y);	Define ou atualiza depot com a expressão à direita. Essa informação participa da rotina de definir a célula que representa o depósito.
86	data[x][y] = DEPOT;	Marca a célula como depósito, substituindo seu conteúdo representado neste vetor.
87	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
89	public void setAgCarryingGold(int ag) {	Inicia o método para incluir o agente no conjunto de portadores.
90	agWithGold.add(ag);	Inclui o identificador do agente no conjunto de portadores.
91	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
92	public void setAgNotCarryingGold(int ag) {	Inicia o método para retirar o agente do conjunto de portadores.
93	agWithGold.remove(ag);	Retira o identificador do conjunto de portadores.
94	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
98	boolean move(Move dir, int ag) throws Exception {	Inicia o método para tentar mudar a posição conforme a direção e a ocupação da célula.
99	Location l = getAgPos(ag);	Define ou atualiza Location l com a expressão à direita. Essa informação participa da rotina de tentar mudar a posição conforme a direção e a ocupação da célula.
100	switch (dir) {	Escolhe um ramo de execução de acordo com o valor indicado.
101	case UP:	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
102	if (isFree(l.x, l.y - 1)) {	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é tentar mudar a posição conforme a direção e a ocupação da célula.
103	setAgPos(ag, l.x, l.y - 1);	Executa a chamada indicada como parte da rotina de tentar mudar a posição conforme a direção e a ocupação da célula. Seus argumentos indicam os dados usados nesta etapa.
104	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
105	break;	Encerra este ramo do switch; evita continuar pelo caso seguinte.
106	case DOWN:	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
107	if (isFree(l.x, l.y + 1)) {	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é tentar mudar a posição conforme a direção e a ocupação da célula.
108	setAgPos(ag, l.x, l.y + 1);	Executa a chamada indicada como parte da rotina de tentar mudar a posição conforme a direção e a ocupação da célula. Seus argumentos indicam os dados usados nesta etapa.

Linha	Código	Como ler
109	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
110	break;	Encerra este ramo do switch; evita continuar pelo caso seguinte.
111	case RIGHT:	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
112	if (isFree(1.x + 1, 1.y)) {	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é tentar mudar a posição conforme a direção e a ocupação da célula.
113	setAgPos(ag, 1.x + 1, 1.y);	Executa a chamada indicada como parte da rotina de tentar mudar a posição conforme a direção e a ocupação da célula. Seus argumentos indicam os dados usados nesta etapa.
114	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
115	break;	Encerra este ramo do switch; evita continuar pelo caso seguinte.
116	case LEFT:	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
117	if (isFree(1.x - 1, 1.y)) {	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é tentar mudar a posição conforme a direção e a ocupação da célula.
118	setAgPos(ag, 1.x - 1, 1.y);	Executa a chamada indicada como parte da rotina de tentar mudar a posição conforme a direção e a ocupação da célula. Seus argumentos indicam os dados usados nesta etapa.
119	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
120	break;	Encerra este ramo do switch; evita continuar pelo caso seguinte.
121	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
122	return true;	O método retorna verdadeiro mesmo se a célula estava bloqueada. Por isso o teste confere a posição, não apenas esse retorno.
123	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
125	boolean pick(int ag) {	Inicia o método para retirar ouro do mapa e registrar a carga quando as condições permitem.
126	Location l = getAgPos(ag);	Define ou atualiza Location l com a expressão à direita. Essa informação participa da rotina de retirar ouro do mapa e registrar a carga quando as condições permitem.
127	if (hasObject(WorldModel.GOLD, 1.x, 1.y)) {	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é retirar ouro do mapa e registrar a carga quando as condições permitem.
128	if (!isCarryingGold(ag)) {	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é retirar ouro do mapa e registrar a carga quando as condições permitem.
129	remove(WorldModel.GOLD, 1.x, 1.y);	Retira o ouro da célula ao efetivar a coleta.
130	setAgCarryingGold(ag);	Registra que o agente passou a carregar a peça retirada do mapa.

Linha	Código	Como ler
131	<code>return true;</code>	Devolve verdadeiro à chamada que está sendo atendida; seu significado depende do método.
132	<code>} else {</code>	Trata a alternativa em que a condição anterior não foi satisfeita.
133	<code>logger.warning("Agent " + (ag + 1) + " is trying the pick gold, but it is already carrying gold!");</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
134	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
135	<code>} else {</code>	Trata a alternativa em que a condição anterior não foi satisfeita.
136	<code>logger.warning("Agent " + (ag + 1) + " is trying the pick gold, but there is no gold at " + l.x + "x" + l.y + "!");</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
137	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
138	<code>return false;</code>	Devolve falso para indicar que a condição ou tentativa não foi satisfeita.
139	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
141	<code>boolean drop(int ag) {</code>	Inicia o método para devolver a carga ao mapa ou contabilizar sua entrega no depósito.
142	<code>Location l = getAgPos(ag);</code>	Define ou atualiza Location l com a expressão à direita. Essa informação participa da rotina de devolver a carga ao mapa ou contabilizar sua entrega no depósito.
143	<code>if (isCarryingGold(ag)) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é devolver a carga ao mapa ou contabilizar sua entrega no depósito.
144	<code>if (l.equals(getDepot())) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é devolver a carga ao mapa ou contabilizar sua entrega no depósito.
145	<code>goldInDepot++;</code>	Conta mais uma unidade entregue. O tutorial ainda mantém esse contador global, sem separação por equipe.
146	<code>logger.info("Agent " + (ag + 1) + " carried a gold to depot!");</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
147	<code>} else {</code>	Trata a alternativa em que a condição anterior não foi satisfeita.
148	<code>add(WorldModel.GOLD, l.x, l.y);</code>	Devolve ouro à célula atual. A regra para uma célula que já contém ouro ainda precisa de revisão antes da competição.
149	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
150	<code>setAgNotCarryingGold(ag);</code>	Registra que o agente deixou de carregar ouro após a operação.
151	<code>return true;</code>	Devolve verdadeiro à chamada que está sendo atendida; seu significado depende do método.
152	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
153	<code>return false;</code>	Devolve falso para indicar que a condição ou tentativa não foi satisfeita.

Linha	Código	Como ler
154	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
195	static WorldModel world1() throws Exception {	Inicia o método para preparar o cenário 1.
196	WorldModel model = WorldModel.create(21, 21, 4);	Cria o mundo com largura, altura e quantidade de agentes indicadas nos argumentos.
197	model.setId("Scenario 1");	Atribui um nome ao cenário para identificá-lo nos registros e na interface.
198	model.setDepot(0, 0);	Define as coordenadas do depósito deste cenário.
199	model.setAgPos(0, 1, 0);	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
200	model.setAgPos(1, 20, 0);	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
201	model.setAgPos(2, 3, 20);	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
202	model.setAgPos(3, 20, 20);	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
203	model.setInitialNbGolds(model.countObjects(WorldModel.GOLD));	Registra a quantidade de recursos usada como referência; na interface, o valor também pode ser atualizado após um clique.
204	return model;	Entrega este resultado a quem chamou a rotina de world1.
205	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
208	static WorldModel world2() throws Exception {	Inicia o método para preparar o cenário 2.
209	WorldModel model = WorldModel.create(35, 35, 4);	Cria o mundo com largura, altura e quantidade de agentes indicadas nos argumentos.
210	model.setId("Scenario 2");	Atribui um nome ao cenário para identificá-lo nos registros e na interface.
211	model.setDepot(0, 0);	Define as coordenadas do depósito deste cenário.
212	model.setAgPos(0, 1, 0);	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
213	model.setAgPos(1, 22, 0);	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
214	model.setAgPos(2, 3, 22);	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
215	model.setAgPos(3, 22, 22);	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
216	model.add(WorldModel.OBSTACLE, 20, 0);	Coloca um obstáculo na célula de coordenadas 20, 0. Esta linha define parte do mapa do cenário.
217	model.add(WorldModel.OBSTACLE, 20, 1);	Coloca um obstáculo na célula de coordenadas 20, 1. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
218	<code>model.add(WorldModel.OBSTACLE, 20, 2);</code>	Coloca um obstáculo na célula de coordenadas 20, 2. Esta linha define parte do mapa do cenário.
219	<code>model.add(WorldModel.OBSTACLE, 20, 3);</code>	Coloca um obstáculo na célula de coordenadas 20, 3. Esta linha define parte do mapa do cenário.
220	<code>model.add(WorldModel.OBSTACLE, 20, 4);</code>	Coloca um obstáculo na célula de coordenadas 20, 4. Esta linha define parte do mapa do cenário.
221	<code>model.add(WorldModel.OBSTACLE, 20, 5);</code>	Coloca um obstáculo na célula de coordenadas 20, 5. Esta linha define parte do mapa do cenário.
222	<code>model.add(WorldModel.OBSTACLE, 20, 6);</code>	Coloca um obstáculo na célula de coordenadas 20, 6. Esta linha define parte do mapa do cenário.
223	<code>model.add(WorldModel.OBSTACLE, 20, 7);</code>	Coloca um obstáculo na célula de coordenadas 20, 7. Esta linha define parte do mapa do cenário.
224	<code>model.add(WorldModel.OBSTACLE, 20, 8);</code>	Coloca um obstáculo na célula de coordenadas 20, 8. Esta linha define parte do mapa do cenário.
225	<code>model.add(WorldModel.OBSTACLE, 20, 9);</code>	Coloca um obstáculo na célula de coordenadas 20, 9. Esta linha define parte do mapa do cenário.
226	<code>model.add(WorldModel.OBSTACLE, 20, 10);</code>	Coloca um obstáculo na célula de coordenadas 20, 10. Esta linha define parte do mapa do cenário.
227	<code>model.add(WorldModel.OBSTACLE, 20, 11);</code>	Coloca um obstáculo na célula de coordenadas 20, 11. Esta linha define parte do mapa do cenário.
228	<code>model.add(WorldModel.OBSTACLE, 20, 12);</code>	Coloca um obstáculo na célula de coordenadas 20, 12. Esta linha define parte do mapa do cenário.
229	<code>model.add(WorldModel.OBSTACLE, 20, 13);</code>	Coloca um obstáculo na célula de coordenadas 20, 13. Esta linha define parte do mapa do cenário.
230	<code>model.add(WorldModel.OBSTACLE, 20, 14);</code>	Coloca um obstáculo na célula de coordenadas 20, 14. Esta linha define parte do mapa do cenário.
231	<code>model.add(WorldModel.OBSTACLE, 20, 15);</code>	Coloca um obstáculo na célula de coordenadas 20, 15. Esta linha define parte do mapa do cenário.
232	<code>model.add(WorldModel.OBSTACLE, 20, 16);</code>	Coloca um obstáculo na célula de coordenadas 20, 16. Esta linha define parte do mapa do cenário.
233	<code>model.add(WorldModel.OBSTACLE, 20, 17);</code>	Coloca um obstáculo na célula de coordenadas 20, 17. Esta linha define parte do mapa do cenário.
234	<code>model.add(WorldModel.OBSTACLE, 20, 18);</code>	Coloca um obstáculo na célula de coordenadas 20, 18. Esta linha define parte do mapa do cenário.
235	<code>model.add(WorldModel.OBSTACLE, 20, 19);</code>	Coloca um obstáculo na célula de coordenadas 20, 19. Esta linha define parte do mapa do cenário.
236	<code>model.add(WorldModel.OBSTACLE, 20, 20);</code>	Coloca um obstáculo na célula de coordenadas 20, 20. Esta linha define parte do mapa do cenário.
237	<code>model.add(WorldModel.OBSTACLE, 21, 20);</code>	Coloca um obstáculo na célula de coordenadas 21, 20. Esta linha define parte do mapa do cenário.
238	<code>model.add(WorldModel.OBSTACLE, 22, 20);</code>	Coloca um obstáculo na célula de coordenadas 22, 20. Esta linha define parte do mapa do cenário.
239	<code>model.add(WorldModel.OBSTACLE, 23, 20);</code>	Coloca um obstáculo na célula de coordenadas 23, 20. Esta linha define parte do mapa do cenário.
240	<code>model.add(WorldModel.OBSTACLE, 24, 20);</code>	Coloca um obstáculo na célula de coordenadas 24, 20. Esta linha define parte do mapa do cenário.
241	<code>model.add(WorldModel.OBSTACLE, 25, 20);</code>	Coloca um obstáculo na célula de coordenadas 25, 20. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
242	<code>model.add(WorldModel.OBSTACLE, 26, 20);</code>	Coloca um obstáculo na célula de coordenadas 26, 20. Esta linha define parte do mapa do cenário.
243	<code>model.add(WorldModel.OBSTACLE, 27, 20);</code>	Coloca um obstáculo na célula de coordenadas 27, 20. Esta linha define parte do mapa do cenário.
244	<code>model.add(WorldModel.OBSTACLE, 28, 20);</code>	Coloca um obstáculo na célula de coordenadas 28, 20. Esta linha define parte do mapa do cenário.
245	<code>model.add(WorldModel.OBSTACLE, 29, 20);</code>	Coloca um obstáculo na célula de coordenadas 29, 20. Esta linha define parte do mapa do cenário.
246	<code>model.add(WorldModel.OBSTACLE, 30, 20);</code>	Coloca um obstáculo na célula de coordenadas 30, 20. Esta linha define parte do mapa do cenário.
247	<code>model.add(WorldModel.OBSTACLE, 31, 20);</code>	Coloca um obstáculo na célula de coordenadas 31, 20. Esta linha define parte do mapa do cenário.
248	<code>model.add(WorldModel.OBSTACLE, 32, 20);</code>	Coloca um obstáculo na célula de coordenadas 32, 20. Esta linha define parte do mapa do cenário.
249	<code>model.add(WorldModel.OBSTACLE, 33, 20);</code>	Coloca um obstáculo na célula de coordenadas 33, 20. Esta linha define parte do mapa do cenário.
250	<code>model.add(WorldModel.OBSTACLE, 34, 20);</code>	Coloca um obstáculo na célula de coordenadas 34, 20. Esta linha define parte do mapa do cenário.
251	<code>model.setInitialNbGolds(model.countObjects(WorldModel.GOLD));</code>	Registra a quantidade de recursos usada como referência; na interface, o valor também pode ser atualizado após um clique.
252	<code>return model;</code>	Entrega este resultado a quem chamou a rotina de world2.
253	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
256	<code>static WorldModel world3() throws Exception {</code>	Inicia o método para preparar o cenário 3.
257	<code>WorldModel model = WorldModel.create(35, 35, 4);</code>	Cria o mundo com largura, altura e quantidade de agentes indicadas nos argumentos.
258	<code>model.setId("Scenario 3");</code>	Atribui um nome ao cenário para identificá-lo nos registros e na interface.
259	<code>model.setDepot(0, 0);</code>	Define as coordenadas do depósito deste cenário.
260	<code>model.setAgPos(0, 1, 0);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
261	<code>model.setAgPos(1, 20, 0);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
262	<code>model.setAgPos(2, 3, 20);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
263	<code>model.setAgPos(3, 20, 20);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
264	<code>model.add(WorldModel.GOLD, 20, 13);</code>	Coloca ouro na célula de coordenadas 20, 13. Esta linha define parte do mapa do cenário.
265	<code>model.add(WorldModel.GOLD, 15, 20);</code>	Coloca ouro na célula de coordenadas 15, 20. Esta linha define parte do mapa do cenário.
266	<code>model.add(WorldModel.GOLD, 1, 1);</code>	Coloca ouro na célula de coordenadas 1, 1. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
267	<code>model.add(WorldModel.GOLD, 3, 5);</code>	Coloca ouro na célula de coordenadas 3, 5. Esta linha define parte do mapa do cenário.
268	<code>model.add(WorldModel.GOLD, 24, 24);</code>	Coloca ouro na célula de coordenadas 24, 24. Esta linha define parte do mapa do cenário.
269	<code>model.add(WorldModel.GOLD, 20, 20);</code>	Coloca ouro na célula de coordenadas 20, 20. Esta linha define parte do mapa do cenário.
270	<code>model.add(WorldModel.GOLD, 26, 21);</code>	Coloca ouro na célula de coordenadas 26, 21. Esta linha define parte do mapa do cenário.
271	<code>model.add(WorldModel.GOLD, 12, 22);</code>	Coloca ouro na célula de coordenadas 12, 22. Esta linha define parte do mapa do cenário.
272	<code>model.add(WorldModel.GOLD, 20, 23);</code>	Coloca ouro na célula de coordenadas 20, 23. Esta linha define parte do mapa do cenário.
273	<code>model.add(WorldModel.GOLD, 33, 24);</code>	Coloca ouro na célula de coordenadas 33, 24. Esta linha define parte do mapa do cenário.
274	<code>model.add(WorldModel.GOLD, 19, 20);</code>	Coloca ouro na célula de coordenadas 19, 20. Esta linha define parte do mapa do cenário.
275	<code>model.add(WorldModel.GOLD, 19, 21);</code>	Coloca ouro na célula de coordenadas 19, 21. Esta linha define parte do mapa do cenário.
276	<code>model.add(WorldModel.GOLD, 34, 34);</code>	Coloca ouro na célula de coordenadas 34, 34. Esta linha define parte do mapa do cenário.
277	<code>model.setInitialNbGolds(model.countObjects(WorldModel.GOLD));</code>	Registra a quantidade de recursos usada como referência; na interface, o valor também pode ser atualizado após um clique.
278	<code>return model;</code>	Entrega este resultado a quem chamou a rotina de world3.
279	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
284	<code>static WorldModel world4() throws Exception {</code>	Inicia o método para preparar o cenário 4.
285	<code>WorldModel model = WorldModel.create(35, 35, 4);</code>	Cria o mundo com largura, altura e quantidade de agentes indicadas nos argumentos.
286	<code>model.setId("Scenario 4");</code>	Atribui um nome ao cenário para identificá-lo nos registros e na interface.
287	<code>model.setDepot(5, 27);</code>	Define as coordenadas do depósito deste cenário.
288	<code>model.setAgPos(0, 1, 0);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
289	<code>model.setAgPos(1, 20, 0);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
290	<code>model.setAgPos(2, 3, 20);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
291	<code>model.setAgPos(3, 20, 20);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
292	<code>model.add(WorldModel.GOLD, 20, 13);</code>	Coloca ouro na célula de coordenadas 20, 13. Esta linha define parte do mapa do cenário.
293	<code>model.add(WorldModel.GOLD, 15, 20);</code>	Coloca ouro na célula de coordenadas 15, 20. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
294	<code>model.add(WorldModel.GOLD, 1, 1);</code>	Coloca ouro na célula de coordenadas 1, 1. Esta linha define parte do mapa do cenário.
295	<code>model.add(WorldModel.GOLD, 3, 5);</code>	Coloca ouro na célula de coordenadas 3, 5. Esta linha define parte do mapa do cenário.
296	<code>model.add(WorldModel.GOLD, 24, 24);</code>	Coloca ouro na célula de coordenadas 24, 24. Esta linha define parte do mapa do cenário.
297	<code>model.add(WorldModel.GOLD, 20, 20);</code>	Coloca ouro na célula de coordenadas 20, 20. Esta linha define parte do mapa do cenário.
298	<code>model.add(WorldModel.GOLD, 20, 21);</code>	Coloca ouro na célula de coordenadas 20, 21. Esta linha define parte do mapa do cenário.
299	<code>model.add(WorldModel.GOLD, 20, 22);</code>	Coloca ouro na célula de coordenadas 20, 22. Esta linha define parte do mapa do cenário.
300	<code>model.add(WorldModel.GOLD, 20, 23);</code>	Coloca ouro na célula de coordenadas 20, 23. Esta linha define parte do mapa do cenário.
301	<code>model.add(WorldModel.GOLD, 20, 24);</code>	Coloca ouro na célula de coordenadas 20, 24. Esta linha define parte do mapa do cenário.
302	<code>model.add(WorldModel.GOLD, 19, 20);</code>	Coloca ouro na célula de coordenadas 19, 20. Esta linha define parte do mapa do cenário.
303	<code>model.add(WorldModel.GOLD, 19, 21);</code>	Coloca ouro na célula de coordenadas 19, 21. Esta linha define parte do mapa do cenário.
304	<code>model.add(WorldModel.GOLD, 34, 34);</code>	Coloca ouro na célula de coordenadas 34, 34. Esta linha define parte do mapa do cenário.
305	<code>model.setInitialNbGolds(model.countObjects(WorldModel.GOLD));</code>	Registra a quantidade de recursos usada como referência; na interface, o valor também pode ser atualizado após um clique.
306	<code>return model;</code>	Entrega este resultado a quem chamou a rotina de world4.
307	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
312	<code>static WorldModel world5() throws Exception {</code>	Inicia o método para preparar o cenário 5.
313	<code>WorldModel model = WorldModel.create(35, 35, 4);</code>	Cria o mundo com largura, altura e quantidade de agentes indicadas nos argumentos.
314	<code>model.setId("Scenario 5");</code>	Atribui um nome ao cenário para identificá-lo nos registros e na interface.
315	<code>model.setDepot(5, 27);</code>	Define as coordenadas do depósito deste cenário.
316	<code>model.setAgPos(0, 1, 0);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
317	<code>model.setAgPos(1, 20, 0);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
318	<code>model.setAgPos(2, 3, 20);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
319	<code>model.setAgPos(3, 20, 20);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
320	<code>model.add(WorldModel.GOLD, 20, 13);</code>	Coloca ouro na célula de coordenadas 20, 13. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
321	<code>model.add(WorldModel.GOLD, 15, 20);</code>	Coloca ouro na célula de coordenadas 15, 20. Esta linha define parte do mapa do cenário.
322	<code>model.add(WorldModel.GOLD, 1, 1);</code>	Coloca ouro na célula de coordenadas 1, 1. Esta linha define parte do mapa do cenário.
323	<code>model.add(WorldModel.GOLD, 3, 5);</code>	Coloca ouro na célula de coordenadas 3, 5. Esta linha define parte do mapa do cenário.
324	<code>model.add(WorldModel.GOLD, 24, 24);</code>	Coloca ouro na célula de coordenadas 24, 24. Esta linha define parte do mapa do cenário.
325	<code>model.add(WorldModel.GOLD, 20, 20);</code>	Coloca ouro na célula de coordenadas 20, 20. Esta linha define parte do mapa do cenário.
326	<code>model.add(WorldModel.GOLD, 20, 21);</code>	Coloca ouro na célula de coordenadas 20, 21. Esta linha define parte do mapa do cenário.
327	<code>model.add(WorldModel.GOLD, 20, 22);</code>	Coloca ouro na célula de coordenadas 20, 22. Esta linha define parte do mapa do cenário.
328	<code>model.add(WorldModel.GOLD, 20, 23);</code>	Coloca ouro na célula de coordenadas 20, 23. Esta linha define parte do mapa do cenário.
329	<code>model.add(WorldModel.GOLD, 20, 24);</code>	Coloca ouro na célula de coordenadas 20, 24. Esta linha define parte do mapa do cenário.
330	<code>model.add(WorldModel.GOLD, 19, 20);</code>	Coloca ouro na célula de coordenadas 19, 20. Esta linha define parte do mapa do cenário.
331	<code>model.add(WorldModel.GOLD, 19, 21);</code>	Coloca ouro na célula de coordenadas 19, 21. Esta linha define parte do mapa do cenário.
332	<code>model.add(WorldModel.GOLD, 34, 34);</code>	Coloca ouro na célula de coordenadas 34, 34. Esta linha define parte do mapa do cenário.
334	<code>model.add(WorldModel.OBSTACLE, 12, 3);</code>	Coloca um obstáculo na célula de coordenadas 12, 3. Esta linha define parte do mapa do cenário.
335	<code>model.add(WorldModel.OBSTACLE, 13, 3);</code>	Coloca um obstáculo na célula de coordenadas 13, 3. Esta linha define parte do mapa do cenário.
336	<code>model.add(WorldModel.OBSTACLE, 14, 3);</code>	Coloca um obstáculo na célula de coordenadas 14, 3. Esta linha define parte do mapa do cenário.
337	<code>model.add(WorldModel.OBSTACLE, 15, 3);</code>	Coloca um obstáculo na célula de coordenadas 15, 3. Esta linha define parte do mapa do cenário.
338	<code>model.add(WorldModel.OBSTACLE, 18, 3);</code>	Coloca um obstáculo na célula de coordenadas 18, 3. Esta linha define parte do mapa do cenário.
339	<code>model.add(WorldModel.OBSTACLE, 19, 3);</code>	Coloca um obstáculo na célula de coordenadas 19, 3. Esta linha define parte do mapa do cenário.
340	<code>model.add(WorldModel.OBSTACLE, 20, 3);</code>	Coloca um obstáculo na célula de coordenadas 20, 3. Esta linha define parte do mapa do cenário.
341	<code>model.add(WorldModel.OBSTACLE, 14, 8);</code>	Coloca um obstáculo na célula de coordenadas 14, 8. Esta linha define parte do mapa do cenário.
342	<code>model.add(WorldModel.OBSTACLE, 15, 8);</code>	Coloca um obstáculo na célula de coordenadas 15, 8. Esta linha define parte do mapa do cenário.
343	<code>model.add(WorldModel.OBSTACLE, 16, 8);</code>	Coloca um obstáculo na célula de coordenadas 16, 8. Esta linha define parte do mapa do cenário.
344	<code>model.add(WorldModel.OBSTACLE, 17, 8);</code>	Coloca um obstáculo na célula de coordenadas 17, 8. Esta linha define parte do mapa do cenário.
345	<code>model.add(WorldModel.OBSTACLE, 19, 8);</code>	Coloca um obstáculo na célula de coordenadas 19, 8. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
346	<code>model.add(WorldModel.OBSTACLE, 20, 8);</code>	Coloca um obstáculo na célula de coordenadas 20, 8. Esta linha define parte do mapa do cenário.
347	<code>model.setInitialNbGolds(model.countObjects(WorldModel.GOLD));</code>	Registra a quantidade de recursos usada como referência; na interface, o valor também pode ser atualizado após um clique.
348	<code>return model;</code>	Entrega este resultado a quem chamou a rotina de world5.
349	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
353	<code>static WorldModel world6() throws Exception {</code>	Inicia o método para preparar o cenário 6.
354	<code>WorldModel model = WorldModel.create(35, 35, 4);</code>	Cria o mundo com largura, altura e quantidade de agentes indicadas nos argumentos.
355	<code>model.setId("Scenario 6");</code>	Atribui um nome ao cenário para identificá-lo nos registros e na interface.
356	<code>model.setDepot(16, 16);</code>	Define as coordenadas do depósito deste cenário.
357	<code>model.setAgPos(0, 1, 0);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
358	<code>model.setAgPos(1, 20, 0);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
359	<code>model.setAgPos(2, 6, 26);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
360	<code>model.setAgPos(3, 20, 20);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
361	<code>model.add(WorldModel.GOLD, 20, 13);</code>	Coloca ouro na célula de coordenadas 20, 13. Esta linha define parte do mapa do cenário.
362	<code>model.add(WorldModel.GOLD, 15, 20);</code>	Coloca ouro na célula de coordenadas 15, 20. Esta linha define parte do mapa do cenário.
363	<code>model.add(WorldModel.GOLD, 1, 1);</code>	Coloca ouro na célula de coordenadas 1, 1. Esta linha define parte do mapa do cenário.
364	<code>model.add(WorldModel.GOLD, 3, 5);</code>	Coloca ouro na célula de coordenadas 3, 5. Esta linha define parte do mapa do cenário.
365	<code>model.add(WorldModel.GOLD, 24, 24);</code>	Coloca ouro na célula de coordenadas 24, 24. Esta linha define parte do mapa do cenário.
366	<code>model.add(WorldModel.GOLD, 20, 20);</code>	Coloca ouro na célula de coordenadas 20, 20. Esta linha define parte do mapa do cenário.
367	<code>model.add(WorldModel.GOLD, 20, 21);</code>	Coloca ouro na célula de coordenadas 20, 21. Esta linha define parte do mapa do cenário.
368	<code>model.add(WorldModel.GOLD, 2, 22);</code>	Coloca ouro na célula de coordenadas 2, 22. Esta linha define parte do mapa do cenário.
369	<code>model.add(WorldModel.GOLD, 2, 12);</code>	Coloca ouro na célula de coordenadas 2, 12. Esta linha define parte do mapa do cenário.
370	<code>model.add(WorldModel.GOLD, 19, 2);</code>	Coloca ouro na célula de coordenadas 19, 2. Esta linha define parte do mapa do cenário.
371	<code>model.add(WorldModel.GOLD, 14, 4);</code>	Coloca ouro na célula de coordenadas 14, 4. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
372	<code>model.add(WorldModel.GOLD, 34, 34);</code>	Coloca ouro na célula de coordenadas 34, 34. Esta linha define parte do mapa do cenário.
374	<code>model.add(WorldModel.OBSTACLE, 12, 3);</code>	Coloca um obstáculo na célula de coordenadas 12, 3. Esta linha define parte do mapa do cenário.
375	<code>model.add(WorldModel.OBSTACLE, 13, 3);</code>	Coloca um obstáculo na célula de coordenadas 13, 3. Esta linha define parte do mapa do cenário.
376	<code>model.add(WorldModel.OBSTACLE, 14, 3);</code>	Coloca um obstáculo na célula de coordenadas 14, 3. Esta linha define parte do mapa do cenário.
377	<code>model.add(WorldModel.OBSTACLE, 15, 3);</code>	Coloca um obstáculo na célula de coordenadas 15, 3. Esta linha define parte do mapa do cenário.
378	<code>model.add(WorldModel.OBSTACLE, 18, 3);</code>	Coloca um obstáculo na célula de coordenadas 18, 3. Esta linha define parte do mapa do cenário.
379	<code>model.add(WorldModel.OBSTACLE, 19, 3);</code>	Coloca um obstáculo na célula de coordenadas 19, 3. Esta linha define parte do mapa do cenário.
380	<code>model.add(WorldModel.OBSTACLE, 20, 3);</code>	Coloca um obstáculo na célula de coordenadas 20, 3. Esta linha define parte do mapa do cenário.
381	<code>model.add(WorldModel.OBSTACLE, 14, 8);</code>	Coloca um obstáculo na célula de coordenadas 14, 8. Esta linha define parte do mapa do cenário.
382	<code>model.add(WorldModel.OBSTACLE, 15, 8);</code>	Coloca um obstáculo na célula de coordenadas 15, 8. Esta linha define parte do mapa do cenário.
383	<code>model.add(WorldModel.OBSTACLE, 16, 8);</code>	Coloca um obstáculo na célula de coordenadas 16, 8. Esta linha define parte do mapa do cenário.
384	<code>model.add(WorldModel.OBSTACLE, 17, 8);</code>	Coloca um obstáculo na célula de coordenadas 17, 8. Esta linha define parte do mapa do cenário.
385	<code>model.add(WorldModel.OBSTACLE, 19, 8);</code>	Coloca um obstáculo na célula de coordenadas 19, 8. Esta linha define parte do mapa do cenário.
386	<code>model.add(WorldModel.OBSTACLE, 20, 8);</code>	Coloca um obstáculo na célula de coordenadas 20, 8. Esta linha define parte do mapa do cenário.
388	<code>model.add(WorldModel.OBSTACLE, 12, 32);</code>	Coloca um obstáculo na célula de coordenadas 12, 32. Esta linha define parte do mapa do cenário.
389	<code>model.add(WorldModel.OBSTACLE, 13, 32);</code>	Coloca um obstáculo na célula de coordenadas 13, 32. Esta linha define parte do mapa do cenário.
390	<code>model.add(WorldModel.OBSTACLE, 14, 32);</code>	Coloca um obstáculo na célula de coordenadas 14, 32. Esta linha define parte do mapa do cenário.
391	<code>model.add(WorldModel.OBSTACLE, 15, 32);</code>	Coloca um obstáculo na célula de coordenadas 15, 32. Esta linha define parte do mapa do cenário.
392	<code>model.add(WorldModel.OBSTACLE, 18, 32);</code>	Coloca um obstáculo na célula de coordenadas 18, 32. Esta linha define parte do mapa do cenário.
393	<code>model.add(WorldModel.OBSTACLE, 19, 32);</code>	Coloca um obstáculo na célula de coordenadas 19, 32. Esta linha define parte do mapa do cenário.
394	<code>model.add(WorldModel.OBSTACLE, 20, 32);</code>	Coloca um obstáculo na célula de coordenadas 20, 32. Esta linha define parte do mapa do cenário.
395	<code>model.add(WorldModel.OBSTACLE, 14, 28);</code>	Coloca um obstáculo na célula de coordenadas 14, 28. Esta linha define parte do mapa do cenário.
396	<code>model.add(WorldModel.OBSTACLE, 15, 28);</code>	Coloca um obstáculo na célula de coordenadas 15, 28. Esta linha define parte do mapa do cenário.
397	<code>model.add(WorldModel.OBSTACLE, 16, 28);</code>	Coloca um obstáculo na célula de coordenadas 16, 28. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
398	<code>model.add(WorldModel.OBSTACLE, 17, 28);</code>	Coloca um obstáculo na célula de coordenadas 17, 28. Esta linha define parte do mapa do cenário.
399	<code>model.add(WorldModel.OBSTACLE, 19, 28);</code>	Coloca um obstáculo na célula de coordenadas 19, 28. Esta linha define parte do mapa do cenário.
400	<code>model.add(WorldModel.OBSTACLE, 20, 28);</code>	Coloca um obstáculo na célula de coordenadas 20, 28. Esta linha define parte do mapa do cenário.
402	<code>model.add(WorldModel.OBSTACLE, 3, 12);</code>	Coloca um obstáculo na célula de coordenadas 3, 12. Esta linha define parte do mapa do cenário.
403	<code>model.add(WorldModel.OBSTACLE, 3, 13);</code>	Coloca um obstáculo na célula de coordenadas 3, 13. Esta linha define parte do mapa do cenário.
404	<code>model.add(WorldModel.OBSTACLE, 3, 14);</code>	Coloca um obstáculo na célula de coordenadas 3, 14. Esta linha define parte do mapa do cenário.
405	<code>model.add(WorldModel.OBSTACLE, 3, 15);</code>	Coloca um obstáculo na célula de coordenadas 3, 15. Esta linha define parte do mapa do cenário.
406	<code>model.add(WorldModel.OBSTACLE, 3, 18);</code>	Coloca um obstáculo na célula de coordenadas 3, 18. Esta linha define parte do mapa do cenário.
407	<code>model.add(WorldModel.OBSTACLE, 3, 19);</code>	Coloca um obstáculo na célula de coordenadas 3, 19. Esta linha define parte do mapa do cenário.
408	<code>model.add(WorldModel.OBSTACLE, 3, 20);</code>	Coloca um obstáculo na célula de coordenadas 3, 20. Esta linha define parte do mapa do cenário.
409	<code>model.add(WorldModel.OBSTACLE, 8, 14);</code>	Coloca um obstáculo na célula de coordenadas 8, 14. Esta linha define parte do mapa do cenário.
410	<code>model.add(WorldModel.OBSTACLE, 8, 15);</code>	Coloca um obstáculo na célula de coordenadas 8, 15. Esta linha define parte do mapa do cenário.
411	<code>model.add(WorldModel.OBSTACLE, 8, 16);</code>	Coloca um obstáculo na célula de coordenadas 8, 16. Esta linha define parte do mapa do cenário.
412	<code>model.add(WorldModel.OBSTACLE, 8, 17);</code>	Coloca um obstáculo na célula de coordenadas 8, 17. Esta linha define parte do mapa do cenário.
413	<code>model.add(WorldModel.OBSTACLE, 8, 19);</code>	Coloca um obstáculo na célula de coordenadas 8, 19. Esta linha define parte do mapa do cenário.
414	<code>model.add(WorldModel.OBSTACLE, 8, 20);</code>	Coloca um obstáculo na célula de coordenadas 8, 20. Esta linha define parte do mapa do cenário.
416	<code>model.add(WorldModel.OBSTACLE, 32, 12);</code>	Coloca um obstáculo na célula de coordenadas 32, 12. Esta linha define parte do mapa do cenário.
417	<code>model.add(WorldModel.OBSTACLE, 32, 13);</code>	Coloca um obstáculo na célula de coordenadas 32, 13. Esta linha define parte do mapa do cenário.
418	<code>model.add(WorldModel.OBSTACLE, 32, 14);</code>	Coloca um obstáculo na célula de coordenadas 32, 14. Esta linha define parte do mapa do cenário.
419	<code>model.add(WorldModel.OBSTACLE, 32, 15);</code>	Coloca um obstáculo na célula de coordenadas 32, 15. Esta linha define parte do mapa do cenário.
420	<code>model.add(WorldModel.OBSTACLE, 32, 18);</code>	Coloca um obstáculo na célula de coordenadas 32, 18. Esta linha define parte do mapa do cenário.
421	<code>model.add(WorldModel.OBSTACLE, 32, 19);</code>	Coloca um obstáculo na célula de coordenadas 32, 19. Esta linha define parte do mapa do cenário.
422	<code>model.add(WorldModel.OBSTACLE, 32, 20);</code>	Coloca um obstáculo na célula de coordenadas 32, 20. Esta linha define parte do mapa do cenário.
423	<code>model.add(WorldModel.OBSTACLE, 28, 14);</code>	Coloca um obstáculo na célula de coordenadas 28, 14. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
424	<code>model.add(WorldModel.OBSTACLE, 28, 15);</code>	Coloca um obstáculo na célula de coordenadas 28, 15. Esta linha define parte do mapa do cenário.
425	<code>model.add(WorldModel.OBSTACLE, 28, 16);</code>	Coloca um obstáculo na célula de coordenadas 28, 16. Esta linha define parte do mapa do cenário.
426	<code>model.add(WorldModel.OBSTACLE, 28, 17);</code>	Coloca um obstáculo na célula de coordenadas 28, 17. Esta linha define parte do mapa do cenário.
427	<code>model.add(WorldModel.OBSTACLE, 28, 19);</code>	Coloca um obstáculo na célula de coordenadas 28, 19. Esta linha define parte do mapa do cenário.
428	<code>model.add(WorldModel.OBSTACLE, 28, 20);</code>	Coloca um obstáculo na célula de coordenadas 28, 20. Esta linha define parte do mapa do cenário.
430	<code>model.add(WorldModel.OBSTACLE, 13, 13);</code>	Coloca um obstáculo na célula de coordenadas 13, 13. Esta linha define parte do mapa do cenário.
431	<code>model.add(WorldModel.OBSTACLE, 13, 14);</code>	Coloca um obstáculo na célula de coordenadas 13, 14. Esta linha define parte do mapa do cenário.
433	<code>model.add(WorldModel.OBSTACLE, 13, 16);</code>	Coloca um obstáculo na célula de coordenadas 13, 16. Esta linha define parte do mapa do cenário.
434	<code>model.add(WorldModel.OBSTACLE, 13, 17);</code>	Coloca um obstáculo na célula de coordenadas 13, 17. Esta linha define parte do mapa do cenário.
436	<code>model.add(WorldModel.OBSTACLE, 13, 19);</code>	Coloca um obstáculo na célula de coordenadas 13, 19. Esta linha define parte do mapa do cenário.
437	<code>model.add(WorldModel.OBSTACLE, 14, 19);</code>	Coloca um obstáculo na célula de coordenadas 14, 19. Esta linha define parte do mapa do cenário.
439	<code>model.add(WorldModel.OBSTACLE, 16, 19);</code>	Coloca um obstáculo na célula de coordenadas 16, 19. Esta linha define parte do mapa do cenário.
440	<code>model.add(WorldModel.OBSTACLE, 17, 19);</code>	Coloca um obstáculo na célula de coordenadas 17, 19. Esta linha define parte do mapa do cenário.
442	<code>model.add(WorldModel.OBSTACLE, 19, 19);</code>	Coloca um obstáculo na célula de coordenadas 19, 19. Esta linha define parte do mapa do cenário.
443	<code>model.add(WorldModel.OBSTACLE, 19, 18);</code>	Coloca um obstáculo na célula de coordenadas 19, 18. Esta linha define parte do mapa do cenário.
445	<code>model.add(WorldModel.OBSTACLE, 19, 16);</code>	Coloca um obstáculo na célula de coordenadas 19, 16. Esta linha define parte do mapa do cenário.
446	<code>model.add(WorldModel.OBSTACLE, 19, 15);</code>	Coloca um obstáculo na célula de coordenadas 19, 15. Esta linha define parte do mapa do cenário.
448	<code>model.add(WorldModel.OBSTACLE, 19, 13);</code>	Coloca um obstáculo na célula de coordenadas 19, 13. Esta linha define parte do mapa do cenário.
449	<code>model.add(WorldModel.OBSTACLE, 18, 13);</code>	Coloca um obstáculo na célula de coordenadas 18, 13. Esta linha define parte do mapa do cenário.
451	<code>model.add(WorldModel.OBSTACLE, 16, 13);</code>	Coloca um obstáculo na célula de coordenadas 16, 13. Esta linha define parte do mapa do cenário.
452	<code>model.add(WorldModel.OBSTACLE, 15, 13);</code>	Coloca um obstáculo na célula de coordenadas 15, 13. Esta linha define parte do mapa do cenário.
455	<code>model.add(WorldModel.OBSTACLE, 2, 32);</code>	Coloca um obstáculo na célula de coordenadas 2, 32. Esta linha define parte do mapa do cenário.
456	<code>model.add(WorldModel.OBSTACLE, 3, 32);</code>	Coloca um obstáculo na célula de coordenadas 3, 32. Esta linha define parte do mapa do cenário.
457	<code>model.add(WorldModel.OBSTACLE, 4, 32);</code>	Coloca um obstáculo na célula de coordenadas 4, 32. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
458	<code>model.add(WorldModel.OBSTACLE, 5, 32);</code>	Coloca um obstáculo na célula de coordenadas 5, 32. Esta linha define parte do mapa do cenário.
459	<code>model.add(WorldModel.OBSTACLE, 6, 32);</code>	Coloca um obstáculo na célula de coordenadas 6, 32. Esta linha define parte do mapa do cenário.
460	<code>model.add(WorldModel.OBSTACLE, 7, 32);</code>	Coloca um obstáculo na célula de coordenadas 7, 32. Esta linha define parte do mapa do cenário.
461	<code>model.add(WorldModel.OBSTACLE, 8, 32);</code>	Coloca um obstáculo na célula de coordenadas 8, 32. Esta linha define parte do mapa do cenário.
462	<code>model.add(WorldModel.OBSTACLE, 9, 32);</code>	Coloca um obstáculo na célula de coordenadas 9, 32. Esta linha define parte do mapa do cenário.
463	<code>model.add(WorldModel.OBSTACLE, 10, 32);</code>	Coloca um obstáculo na célula de coordenadas 10, 32. Esta linha define parte do mapa do cenário.
464	<code>model.add(WorldModel.OBSTACLE, 10, 31);</code>	Coloca um obstáculo na célula de coordenadas 10, 31. Esta linha define parte do mapa do cenário.
465	<code>model.add(WorldModel.OBSTACLE, 10, 30);</code>	Coloca um obstáculo na célula de coordenadas 10, 30. Esta linha define parte do mapa do cenário.
466	<code>model.add(WorldModel.OBSTACLE, 10, 29);</code>	Coloca um obstáculo na célula de coordenadas 10, 29. Esta linha define parte do mapa do cenário.
467	<code>model.add(WorldModel.OBSTACLE, 10, 28);</code>	Coloca um obstáculo na célula de coordenadas 10, 28. Esta linha define parte do mapa do cenário.
468	<code>model.add(WorldModel.OBSTACLE, 10, 27);</code>	Coloca um obstáculo na célula de coordenadas 10, 27. Esta linha define parte do mapa do cenário.
469	<code>model.add(WorldModel.OBSTACLE, 10, 26);</code>	Coloca um obstáculo na célula de coordenadas 10, 26. Esta linha define parte do mapa do cenário.
470	<code>model.add(WorldModel.OBSTACLE, 10, 25);</code>	Coloca um obstáculo na célula de coordenadas 10, 25. Esta linha define parte do mapa do cenário.
471	<code>model.add(WorldModel.OBSTACLE, 10, 24);</code>	Coloca um obstáculo na célula de coordenadas 10, 24. Esta linha define parte do mapa do cenário.
472	<code>model.add(WorldModel.OBSTACLE, 10, 23);</code>	Coloca um obstáculo na célula de coordenadas 10, 23. Esta linha define parte do mapa do cenário.
473	<code>model.add(WorldModel.OBSTACLE, 2, 23);</code>	Coloca um obstáculo na célula de coordenadas 2, 23. Esta linha define parte do mapa do cenário.
474	<code>model.add(WorldModel.OBSTACLE, 3, 23);</code>	Coloca um obstáculo na célula de coordenadas 3, 23. Esta linha define parte do mapa do cenário.
475	<code>model.add(WorldModel.OBSTACLE, 4, 23);</code>	Coloca um obstáculo na célula de coordenadas 4, 23. Esta linha define parte do mapa do cenário.
476	<code>model.add(WorldModel.OBSTACLE, 5, 23);</code>	Coloca um obstáculo na célula de coordenadas 5, 23. Esta linha define parte do mapa do cenário.
477	<code>model.add(WorldModel.OBSTACLE, 6, 23);</code>	Coloca um obstáculo na célula de coordenadas 6, 23. Esta linha define parte do mapa do cenário.
478	<code>model.add(WorldModel.OBSTACLE, 7, 23);</code>	Coloca um obstáculo na célula de coordenadas 7, 23. Esta linha define parte do mapa do cenário.
479	<code>model.add(WorldModel.OBSTACLE, 8, 23);</code>	Coloca um obstáculo na célula de coordenadas 8, 23. Esta linha define parte do mapa do cenário.
480	<code>model.add(WorldModel.OBSTACLE, 9, 23);</code>	Coloca um obstáculo na célula de coordenadas 9, 23. Esta linha define parte do mapa do cenário.
481	<code>model.add(WorldModel.OBSTACLE, 2, 29);</code>	Coloca um obstáculo na célula de coordenadas 2, 29. Esta linha define parte do mapa do cenário.

Linha	Código	Como ler
482	<code>model.add(WorldModel.OBSTACLE, 2, 28);</code>	Coloca um obstáculo na célula de coordenadas 2, 28. Esta linha define parte do mapa do cenário.
483	<code>model.add(WorldModel.OBSTACLE, 2, 27);</code>	Coloca um obstáculo na célula de coordenadas 2, 27. Esta linha define parte do mapa do cenário.
484	<code>model.add(WorldModel.OBSTACLE, 2, 26);</code>	Coloca um obstáculo na célula de coordenadas 2, 26. Esta linha define parte do mapa do cenário.
485	<code>model.add(WorldModel.OBSTACLE, 2, 25);</code>	Coloca um obstáculo na célula de coordenadas 2, 25. Esta linha define parte do mapa do cenário.
486	<code>model.add(WorldModel.OBSTACLE, 2, 24);</code>	Coloca um obstáculo na célula de coordenadas 2, 24. Esta linha define parte do mapa do cenário.
487	<code>model.add(WorldModel.OBSTACLE, 2, 23);</code>	Coloca um obstáculo na célula de coordenadas 2, 23. Esta linha define parte do mapa do cenário.
488	<code>model.add(WorldModel.OBSTACLE, 2, 29);</code>	Coloca um obstáculo na célula de coordenadas 2, 29. Esta linha define parte do mapa do cenário.
489	<code>model.add(WorldModel.OBSTACLE, 3, 29);</code>	Coloca um obstáculo na célula de coordenadas 3, 29. Esta linha define parte do mapa do cenário.
490	<code>model.add(WorldModel.OBSTACLE, 4, 29);</code>	Coloca um obstáculo na célula de coordenadas 4, 29. Esta linha define parte do mapa do cenário.
491	<code>model.add(WorldModel.OBSTACLE, 5, 29);</code>	Coloca um obstáculo na célula de coordenadas 5, 29. Esta linha define parte do mapa do cenário.
492	<code>model.add(WorldModel.OBSTACLE, 6, 29);</code>	Coloca um obstáculo na célula de coordenadas 6, 29. Esta linha define parte do mapa do cenário.
493	<code>model.add(WorldModel.OBSTACLE, 7, 29);</code>	Coloca um obstáculo na célula de coordenadas 7, 29. Esta linha define parte do mapa do cenário.
494	<code>model.add(WorldModel.OBSTACLE, 7, 28);</code>	Coloca um obstáculo na célula de coordenadas 7, 28. Esta linha define parte do mapa do cenário.
495	<code>model.add(WorldModel.OBSTACLE, 7, 27);</code>	Coloca um obstáculo na célula de coordenadas 7, 27. Esta linha define parte do mapa do cenário.
496	<code>model.add(WorldModel.OBSTACLE, 7, 26);</code>	Coloca um obstáculo na célula de coordenadas 7, 26. Esta linha define parte do mapa do cenário.
497	<code>model.add(WorldModel.OBSTACLE, 7, 25);</code>	Coloca um obstáculo na célula de coordenadas 7, 25. Esta linha define parte do mapa do cenário.
498	<code>model.add(WorldModel.OBSTACLE, 6, 25);</code>	Coloca um obstáculo na célula de coordenadas 6, 25. Esta linha define parte do mapa do cenário.
499	<code>model.add(WorldModel.OBSTACLE, 5, 25);</code>	Coloca um obstáculo na célula de coordenadas 5, 25. Esta linha define parte do mapa do cenário.
500	<code>model.add(WorldModel.OBSTACLE, 4, 25);</code>	Coloca um obstáculo na célula de coordenadas 4, 25. Esta linha define parte do mapa do cenário.
501	<code>model.add(WorldModel.OBSTACLE, 4, 26);</code>	Coloca um obstáculo na célula de coordenadas 4, 26. Esta linha define parte do mapa do cenário.
502	<code>model.add(WorldModel.OBSTACLE, 4, 27);</code>	Coloca um obstáculo na célula de coordenadas 4, 27. Esta linha define parte do mapa do cenário.
503	<code>model.setInitialNbGolds(model.countObjects(WorldModel.GOLD));</code>	Registra a quantidade de recursos usada como referência; na interface, o valor também pode ser atualizado após um clique.
504	<code>return model;</code>	Entrega este resultado a quem chamou a rotina de world6.
505	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

Linha	Código	Como ler
507	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

src/env/mining/WorldView.java

Desenha o mapa e oferece controles visuais. A interface foi preservada, mas as verificações registradas usaram execução sem janela.

[Abrir arquivo original](#) · 210 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	package mining;	Agrupar a classe no pacote mining, usado para localizá-la no projeto.
3	import jason.environment.grid.GridWorldView;	Fornecer a janela e a superfície de desenho para um mundo em grade.
5	import java.awt.BorderLayout;	Distribuir componentes entre regiões da janela.
6	import java.awt.Color;	Representar as cores usadas para distinguir objetos e estados.
7	import java.awt.Component;	Representar um elemento visual da interface.
8	import java.awt.FlowLayout;	Organizar componentes em sequência dentro de um painel.
9	import java.awt.Graphics;	Oferecer as operações de desenho usadas pela visualização.
10	import java.awt.event.MouseEvent;	Conter as informações de um evento do mouse.
11	import java.awt.event.MouseListener;	Definir os tratamentos para clique e outros eventos do mouse.
12	import java.awt.event.MouseMotionListener;	Definir os tratamentos para deslocamento e arrasto do mouse.
13	import java.util.Hashtable;	Relacionar valores do controle deslizante aos rótulos exibidos.
15	import javax.swing.BorderFactory;	Criar bordas para separar os painéis da interface.
16	import javax.swing.BoxLayout;	Organizar componentes em uma linha ou coluna.
17	import javax.swing.JLabel;	Exibir um texto curto, como posição do mouse ou total depositado.
18	import javax.swing.JPanel;	Agrupar componentes da janela.
19	import javax.swing.JSlider;	Oferecer o controle deslizante da velocidade de execução.
20	import javax.swing.event.ChangeEvent;	Representar uma mudança em um componente, como o controle de velocidade.
21	import javax.swing.event.ChangeListener;	Permitir reagir às mudanças do controle de velocidade.
24	@SuppressWarnings("serial")	Dispensa o aviso de serialização herdado da interface Swing; não altera as regras da simulação.
25	public class WorldView extends GridWorldView {	Definir uma classe e, quando indicado por extends ou implements, relacionar seu comportamento à base ou às interfaces utilizadas.
27	MiningPlanet env = null;	Configurar ou desenhar um elemento da interface nesta rotina de visualização. A lógica do mundo permanece no modelo.
29	public WorldView(WorldModel model) {	Iniciar o método para WorldView.
30	super(model, "Mining World", 600);	Configurar ou desenhar um elemento da interface nesta rotina de WorldView. A lógica do mundo permanece no modelo.

Linha	Código	Como ler
31	<code>setVisible(true);</code>	Configura ou desenha um elemento da interface nesta rotina de <code>WorldView</code> . A lógica do mundo permanece no modelo.
32	<code>repaint();</code>	Configura ou desenha um elemento da interface nesta rotina de <code>WorldView</code> . A lógica do mundo permanece no modelo.
33	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
35	<code>public void setEnv(MiningPlanet env) {</code>	Inicia o método para associar a visualização ao artefato do ambiente.
36	<code> this.env = env;</code>	Configura ou desenha um elemento da interface nesta rotina de associar a visualização ao artefato do ambiente. A lógica do mundo permanece no modelo.
38	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
40	<code>JLabel jMouseLoc;</code>	Configura ou desenha um elemento da interface nesta rotina de associar a visualização ao artefato do ambiente. A lógica do mundo permanece no modelo.
42	<code>JSlider jSpeed;</code>	Configura ou desenha um elemento da interface nesta rotina de associar a visualização ao artefato do ambiente. A lógica do mundo permanece no modelo.
43	<code>JLabel jGoldsC;</code>	Configura ou desenha um elemento da interface nesta rotina de associar a visualização ao artefato do ambiente. A lógica do mundo permanece no modelo.
45	<code>@Override</code>	Indica que o método redefine um comportamento previsto na classe ou interface de origem.
46	<code>public void initComponents(int width) {</code>	Inicia o método para montar painéis, rótulos, controle de velocidade e eventos da interface.
47	<code> super.initComponents(width);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
48	<code> JPanel args = new JPanel();</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
49	<code> args.setLayout(new BoxLayout(args, BoxLayout.Y_AXIS));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
60	<code> jSpeed = new JSlider();</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
61	<code> jSpeed.setMinimum(0);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
62	<code> jSpeed.setMaximum(400);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
63	<code> jSpeed.setValue(50);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
64	<code> jSpeed.setPaintTicks(true);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.

Linha	Código	Como ler
65	<code>jSpeed.setPaintLabels(true);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
66	<code>jSpeed.setMajorTickSpacing(100);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
67	<code>jSpeed.setMinorTickSpacing(20);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
68	<code>jSpeed.setInverted(true);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
69	<code>Hashtable<Integer,Component> labelTable = new Hashtable<Integer,Component>();</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
70	<code>labelTable.put(0, new JLabel("max"));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
71	<code>labelTable.put(200, new JLabel("speed"));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
72	<code>labelTable.put(400, new JLabel("min"));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
73	<code>jSpeed.setLabelTable(labelTable);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
74	<code>JPanel p = new JPanel(new FlowLayout());</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
75	<code>p.setBorder(BorderFactory.createEtchedBorder());</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
76	<code>p.add(jSpeed);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
79	<code>args.add(p);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
81	<code>JPanel msg = new JPanel();</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
82	<code>msg.setLayout(new BorderLayout(msg, BorderLayout.Y_AXIS));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
83	<code>msg.setBorder(BorderFactory.createEtchedBorder());</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
85	<code>p = new JPanel(new FlowLayout(FlowLayout.CENTER));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.

Linha	Código	Como ler
86	<code>p.add(new JLabel("Click on the cells to add new pieces of gold."));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
87	<code>msg.add(p);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
88	<code>p = new JPanel(new FlowLayout(FlowLayout.CENTER));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
89	<code>p.add(new JLabel("(mouse at:"));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
90	<code>jlMouseLoc = new JLabel("0,0");</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
91	<code>p.add(jlMouseLoc);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
92	<code>msg.add(p);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
93	<code>p = new JPanel(new FlowLayout(FlowLayout.CENTER));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
94	<code>p.add(new JLabel("Collected golds:"));</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
95	<code>jGoldsC = new JLabel("0");</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
96	<code>p.add(jGoldsC);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
97	<code>msg.add(p);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
99	<code>JPanel s = new JPanel(new BorderLayout());</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
100	<code>s.add(BorderLayout.WEST, args);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
101	<code>s.add(BorderLayout.CENTER, msg);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
102	<code>getContentPane().add(BorderLayout.SOUTH, s);</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.
105	<code>jSpeed.addChangeListener(new ChangeListener() {</code>	Configura ou desenha um elemento da interface nesta rotina de montar painéis, rótulos, controle de velocidade e eventos da interface. A lógica do mundo permanece no modelo.

Linha	Código	Como ler
106	<code>public void stateChanged(ChangeEvent e) {</code>	Inicia o método para aplicar a velocidade escolhida no controle visual.
107	<code>if (env != null) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é aplicar a velocidade escolhida no controle visual.
108	<code>env.setSleep((int)jSpeed.getValue());</code>	Configura ou desenha um elemento da interface nesta rotina de aplicar a velocidade escolhida no controle visual. A lógica do mundo permanece no modelo.
109	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
110	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
111	<code>});</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
123	<code>getCanvas().addMouseListener(new MouseListener() {</code>	Configura ou desenha um elemento da interface nesta rotina de aplicar a velocidade escolhida no controle visual. A lógica do mundo permanece no modelo.
124	<code>public void mouseClicked(MouseEvent e) {</code>	Inicia o método para transformar um clique no mapa em inclusão de ouro para estudo.
125	<code>int col = e.getX() / cellSizeW;</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
126	<code>int lin = e.getY() / cellSizeH;</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
127	<code>if (col >= 0 && lin >= 0 && col < getModel().getWidth() && lin < getModel().getHeight()) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é transformar um clique no mapa em inclusão de ouro para estudo.
128	<code>WorldModel wm = (WorldModel)model;</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
129	<code>wm.add(WorldModel.GOLD, col, lin);</code>	Coloca ouro na célula de coordenadas col, lin. Esta linha define parte do mapa do cenário.
130	<code>wm.setInitialNbGolds(wm.getInitialNbGolds()+1);</code>	Registra a quantidade de recursos usada como referência; na interface, o valor também pode ser atualizado após um clique.
131	<code>update(col, lin);</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
132	<code>udpateCollectedGolds();</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
133	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
134	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
135	<code>public void mouseExited(MouseEvent e) {}</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
136	<code>public void mouseEntered(MouseEvent e) {}</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.

Linha	Código	Como ler
137	<code>public void mousePressed(MouseEvent e) {}</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
138	<code>public void mouseReleased(MouseEvent e) {}</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
139	<code>});</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
141	<code>getCanvas().addMouseMotionListener(new MouseMotionListener() {</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
142	<code>public void mouseDragged(MouseEvent e) { }</code>	Configura ou desenha um elemento da interface nesta rotina de transformar um clique no mapa em inclusão de ouro para estudo. A lógica do mundo permanece no modelo.
143	<code>public void mouseMoved(MouseEvent e) {</code>	Inicia o método para mostrar a célula sobre a qual o cursor está.
144	<code>int col = e.getX() / cellSizeW;</code>	Configura ou desenha um elemento da interface nesta rotina de mostrar a célula sobre a qual o cursor está. A lógica do mundo permanece no modelo.
145	<code>int lin = e.getY() / cellSizeH;</code>	Configura ou desenha um elemento da interface nesta rotina de mostrar a célula sobre a qual o cursor está. A lógica do mundo permanece no modelo.
146	<code>if (col >= 0 && lin >= 0 && col < getModel().getWidth() && lin < getModel().getHeight()) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é mostrar a célula sobre a qual o cursor está.
147	<code>j1MouseLoc.setText(col+", "+lin+"");</code>	Configura ou desenha um elemento da interface nesta rotina de mostrar a célula sobre a qual o cursor está. A lógica do mundo permanece no modelo.
148	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
149	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
150	<code>});</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
151	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
153	<code>public void updateCollectedGolds() {</code>	Inicia o método para atualizar o rótulo visual com o total depositado e o total inicial.
154	<code>WorldModel wm = (WorldModel)model;</code>	Configura ou desenha um elemento da interface nesta rotina de atualizar o rótulo visual com o total depositado e o total inicial. A lógica do mundo permanece no modelo.
155	<code>jGoldsC.setText(wm.getGoldsInDepot() + "/" + wm.getInitialNbGolds());</code>	Configura ou desenha um elemento da interface nesta rotina de atualizar o rótulo visual com o total depositado e o total inicial. A lógica do mundo permanece no modelo.
156	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
158	<code>@Override</code>	Indica que o método redefine um comportamento previsto na classe ou interface de origem.
159	<code>public void draw(Graphics g, int x, int y, int object) {</code>	Inicia o método para escolher como desenhar cada tipo de objeto.

Linha	Código	Como ler
160	<code>switch (object) {</code>	Escolhe um ramo de execução de acordo com o valor indicado.
161	<code>case WorldModel.DEPOT: drawDepot(g, x, y); break;</code>	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
162	<code>case WorldModel.GOLD: drawGold(g, x, y); break;</code>	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
163	<code>case WorldModel.ENEMY: drawEnemy(g, x, y); break;</code>	Trata especificamente este valor da seleção. O break, quando presente, encerra o ramo.
164	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
165	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
167	<code>@Override</code>	Indica que o método redefine um comportamento previsto na classe ou interface de origem.
168	<code>public void drawAgent(Graphics g, int x, int y, Color c, int id) {</code>	Inicia o método para desenhar o agente com indicação visual de carga.
169	<code>Color idColor = Color.black;</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o agente com indicação visual de carga. A lógica do mundo permanece no modelo.
170	<code>if (((WorldModel)model).isCarryingGold(id)) {</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é desenhar o agente com indicação visual de carga.
171	<code>super.drawAgent(g, x, y, Color.yellow, -1);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o agente com indicação visual de carga. A lógica do mundo permanece no modelo.
172	<code>} else {</code>	Trata a alternativa em que a condição anterior não foi satisfeita.
173	<code>super.drawAgent(g, x, y, c, -1);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o agente com indicação visual de carga. A lógica do mundo permanece no modelo.
174	<code>idColor = Color.white;</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o agente com indicação visual de carga. A lógica do mundo permanece no modelo.
175	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
176	<code>g.setColor(idColor);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o agente com indicação visual de carga. A lógica do mundo permanece no modelo.
177	<code>drawString(g, x, y, defaultFont, String.valueOf(id+1));</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o agente com indicação visual de carga. A lógica do mundo permanece no modelo.
178	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
180	<code>public void drawDepot(Graphics g, int x, int y) {</code>	Inicia o método para desenhar o depósito na célula correspondente.
181	<code>g.setColor(Color.gray);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o depósito na célula correspondente. A lógica do mundo permanece no modelo.

Linha	Código	Como ler
182	<code>g.fillRect(x * cellSizeW, y * cellSizeH, cellSizeW, cellSizeH);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o depósito na célula correspondente. A lógica do mundo permanece no modelo.
183	<code>g.setColor(Color.pink);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o depósito na célula correspondente. A lógica do mundo permanece no modelo.
184	<code>g.drawRect(x * cellSizeW + 2, y * cellSizeH + 2, cellSizeW - 4, cellSizeH - 4);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o depósito na célula correspondente. A lógica do mundo permanece no modelo.
185	<code>g.drawLine(x * cellSizeW + 2, y * cellSizeH + 2, (x + 1) * cellSizeW - 2, (y + 1) * cellSizeH - 2);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o depósito na célula correspondente. A lógica do mundo permanece no modelo.
186	<code>g.drawLine(x * cellSizeW + 2, (y + 1) * cellSizeH - 2, (x + 1) * cellSizeW - 2, y * cellSizeH + 2);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o depósito na célula correspondente. A lógica do mundo permanece no modelo.
187	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
189	<code>public void drawGold(Graphics g, int x, int y) {</code>	Inicia o método para desenhar o símbolo do ouro.
190	<code>g.setColor(Color.yellow);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
191	<code>g.drawRect(x * cellSizeW + 2, y * cellSizeH + 2, cellSizeW - 4, cellSizeH - 4);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
192	<code>int[] vx = new int[4];</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
193	<code>int[] vy = new int[4];</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
194	<code>vx[0] = x * cellSizeW + (cellSizeW / 2);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
195	<code>vy[0] = y * cellSizeH;</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
196	<code>vx[1] = (x + 1) * cellSizeW;</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
197	<code>vy[1] = y * cellSizeH + (cellSizeH / 2);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
198	<code>vx[2] = x * cellSizeW + (cellSizeW / 2);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
199	<code>vy[2] = (y + 1) * cellSizeH;</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
200	<code>vx[3] = x * cellSizeW;</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.

Linha	Código	Como ler
201	<code>vy[3] = y * cellSizeH + (cellSizeH / 2);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
202	<code>g.fillPolygon(vx, vy, 4);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo do ouro. A lógica do mundo permanece no modelo.
203	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
205	<code>public void drawEnemy(Graphics g, int x, int y) {</code>	Inicia o método para desenhar o símbolo de outro agente segundo a convenção visual.
206	<code>g.setColor(Color.red);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo de outro agente segundo a convenção visual. A lógica do mundo permanece no modelo.
207	<code>g.fillOval(x * cellSizeW + 7, y * cellSizeH + 7, cellSizeW - 8, cellSizeH - 8);</code>	Configura ou desenha um elemento da interface nesta rotina de desenhar o símbolo de outro agente segundo a convenção visual. A lógica do mundo permanece no modelo.
208	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
210	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

src/test/java/mining/ModelChecks.java

Verifica situações pequenas do modelo, como ocupação, capacidade, devolução e conservação. Falhas levantam um erro, em vez de apenas imprimir uma mensagem.

[Abrir arquivo original](#) · 36 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>package mining;</code>	Agrupa a classe no pacote mining, usado para localizá-la no projeto.
4	<code>public class ModelChecks {</code>	Define uma classe e, quando indicado por extends ou implements, relaciona seu comportamento à base ou às interfaces utilizadas.
5	<code>static int checks = 0;</code>	Define ou atualiza static int checks com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
6	<code>static void check(boolean condition, String message) {</code>	Inicia o método para interromper a verificação quando uma condição esperada não é satisfeita.
7	<code>if (!condition) throw new AssertionError(message);</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
8	<code>checks++;</code>	Executa a chamada indicada como parte da rotina de interromper a verificação quando uma condição esperada não é satisfeita. Seus argumentos indicam os dados usados nesta etapa.
9	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
10	<code>public static void main(String[] args) throws Exception {</code>	Inicia o método para executar a sequência de verificações do modelo.

Linha	Código	Como ler
11	<code>WorldModel.destroy();</code>	Executa a chamada indicada como parte da rotina de executar a sequência de verificações do modelo. Seus argumentos indicam os dados usados nesta etapa.
12	<code>WorldModel w = WorldModel.create(4,4,2);</code>	Cria o mundo com largura, altura e quantidade de agentes indicadas nos argumentos.
13	<code>w.setDepot(0,0);</code>	Define as coordenadas do depósito deste cenário.
14	<code>w.setAgPos(0,1,1);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
15	<code>w.setAgPos(1,2,1);</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
16	<code>w.add(WorldModel.GOLD,1,1);</code>	Coloca ouro na célula de coordenadas 1, 1. Esta linha define parte do mapa do cenário.
17	<code>w.add(WorldModel.GOLD,1,2);</code>	Coloca ouro na célula de coordenadas 1, 2. Esta linha define parte do mapa do cenário.
18	<code>w.add(WorldModel.OBSTACLE,0,1);</code>	Coloca um obstáculo na célula de coordenadas 0, 1. Esta linha define parte do mapa do cenário.
19	<code>w.move(MiningPlanet.Move.LEFT,0);</code>	Executa a chamada indicada como parte da rotina de executar a sequência de verificações do modelo. Seus argumentos indicam os dados usados nesta etapa.
20	<code>check(w.getAgPos(0).x==1 && w.getAgPos(0).y==1, "atravessou obstaculo");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
21	<code>w.move(MiningPlanet.Move.RIGHT,0);</code>	Executa a chamada indicada como parte da rotina de executar a sequência de verificações do modelo. Seus argumentos indicam os dados usados nesta etapa.
22	<code>check(w.getAgPos(0).x==1, "sobreposicao de agentes");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
23	<code>check(w.pick(0) && w.isCarryingGold(0), "coleta nao ocorreu");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
24	<code>check(!w.hasObject(WorldModel.GOLD,1,1), "ouro duplicado apos coleta");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
25	<code>w.move(MiningPlanet.Move.DOWN,0);</code>	Executa a chamada indicada como parte da rotina de executar a sequência de verificações do modelo. Seus argumentos indicam os dados usados nesta etapa.
26	<code>check(!w.pick(0) && w.hasObject(WorldModel.GOLD,1,2), "capacidade excedida");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
27	<code>w.move(MiningPlanet.Move.RIGHT,0);</code>	Executa a chamada indicada como parte da rotina de executar a sequência de verificações do modelo. Seus argumentos indicam os dados usados nesta etapa.
28	<code>check(w.drop(0) && w.getGoldsInDepot()==0 && w.hasObject(WorldModel.GOLD,2,2), "descarte fora do deposito");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
29	<code>check(w.pick(0), "nova coleta");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.

Linha	Código	Como ler
30	<code>w.setAgPos(0,0,0); // fixture coloca o agente no destino; a integracao testa o caminho real.</code>	Define a posição do agente indicado. Nos cenários, os identificadores começam em zero; durante movimentos, a alteração ocorre após testar a célula.
31	<code>check(w.drop(0) && w.getGoldsInDepot()==1 && !w.isCarryingGold(0),"deposito incorreto");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
32	<code>check(!w.drop(0) && w.getGoldsInDepot()==1,"pontuacao duplicada");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
33	<code>check(w.countObjects(WorldModel.GOLD)+w.getGoldsInDepot()==2,"conservacao do ouro");</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
34	<code>System.out.println("GOLD_MINERS MODEL_CHECKS passed="+checks);</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
35	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
36	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

src/test/java/mining/ValidationProbe.java

Observador usado somente no teste. Confere o estado real após a entrega, sem orientar a escolha de ações dos mineradores.

[Abrir arquivo original](#) · 19 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>package mining;</code>	Agrupa a classe no pacote mining, usado para localizá-la no projeto.
3	<code>import cartago.Artifact;</code>	Fornece a base para construir um artefato CArtaGO.
4	<code>import cartago.OPERATION;</code>	Marca um método como operação do artefato acessível aos agentes.
7	<code>public class ValidationProbe extends Artifact {</code>	Define uma classe e, quando indicado por extends ou implements, relaciona seu comportamento à base ou às interfaces utilizadas.
8	<code>void init() {}</code>	Executa a chamada indicada como parte da rotina de preparação deste arquivo. Seus argumentos indicam os dados usados nesta etapa.
10	<code>@OPERATION void validateDelivery() {</code>	Expõe uma operação do artefato aos agentes; as condições e o efeito ficam no corpo indicado.
11	<code>WorldModel world = WorldModel.get();</code>	Define ou atualiza WorldModel world com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
12	<code>if (world == null world.getGoldsInDepot() != 1 world.isCarryingGold(0)</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é preparação deste arquivo.
13	<code> world.countObjects(WorldModel.GOLD) != world.getInitialNbGolds()-1) {</code>	Define ou atualiza world.countObjects(WorldModel.GOLD) ! com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.

Linha	Código	Como ler
14	<code>failed("A entrega nao corresponde ao estado do mundo");</code>	Executa a chamada indicada como parte da rotina de preparação deste arquivo. Seus argumentos indicam os dados usados nesta etapa.
15	<code>return;</code>	Executa a chamada indicada como parte da rotina de preparação deste arquivo. Seus argumentos indicam os dados usados nesta etapa.
16	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
17	<code>System.out.println("GOLD_MINERS WORLD_VERIFIED deposited=1 carrying=false conservation=ok");</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
18	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
19	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

scripts/laboratorio.py

Prepara dependências, usa o Java adequado, compila e controla processos com prazo máximo. Distingue estudo, verificação e futura competição.

[Abrir arquivo original](#) · 162 linhas · numeração da cópia documentada

Linha	Código	Como ler
2	<code>from pathlib import Path</code>	Trabalha com caminhos e arquivos sem montar separadores manualmente.
3	<code>from datetime import datetime, timezone</code>	Registra a data e a hora das evidências. Permite identificar explicitamente horários em UTC.
4	<code>import argparse</code>	Lê comandos e opções digitados no terminal, com ajuda e validação de argumentos.
5	<code>import hashlib</code>	Calcula hashes para reconhecer a versão exata de um arquivo.
6	<code>import json</code>	Lê e grava configurações e resultados em JSON.
7	<code>import os</code>	Consulta informações do sistema e prepara cópias das variáveis do processo.
8	<code>import re</code>	Reconhece padrões de texto, como mensagens de erro nos registros.
9	<code>import shutil</code>	Copia arquivos e procura programas disponíveis no ambiente.
10	<code>import subprocess</code>	Inicia e acompanha Java e Gradle sem executar comandos de outros processos.
11	<code>import sys</code>	Disponibiliza informações do interpretador Python; conferir se o recurso é necessário antes de mantê-lo numa revisão.
12	<code>import io</code>	Permite ler o ZIP baixado diretamente da memória.
13	<code>import urllib.request</code>	Obtém a dependência do endereço oficial indicado no programa.
14	<code>import zipfile</code>	Lê o pacote ZIP e seleciona somente a biblioteca necessária.
16	<code>ROOT = Path(__file__).resolve().parents[1]</code>	Define ou atualiza ROOT com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.

Linha	Código	Como ler
18	<code>def prepare_search_library():</code>	Define a função para obter a biblioteca do tutorial e conferir sua integridade. Os parâmetros são os dados que a chamada recebe.
20	<code>target = ROOT/'lib/search.jar'</code>	Define ou atualiza target com a expressão à direita. Essa informação participa da rotina de obter a biblioteca do tutorial e conferir sua integridade.
21	<code>expected = '290e21c7300c74e16f1eb620f92bbcd5c3a87adf2f9e6a5d8f595c0b9ae2b34e'</code>	Define ou atualiza expected com a expressão à direita. Essa informação participa da rotina de obter a biblioteca do tutorial e conferir sua integridade.
22	<code>if not target.exists():</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é obter a biblioteca do tutorial e conferir sua integridade.
23	<code>url = 'https://raw.githubusercontent.com/jacamo- lang/jacamo/main/doc/tutorials/gold- miners/initial-gold-miners.zip'</code>	Define ou atualiza url com a expressão à direita. Essa informação participa da rotina de obter a biblioteca do tutorial e conferir sua integridade.
24	<code>print('Obtendo search.jar do pacote oficial JaCaMo...')</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
25	<code>with urllib.request.urlopen(url,timeout=45) as response: data = response.read()</code>	Abre um recurso dentro de um bloco que cuida de fechá-lo ao terminar, inclusive se houver erro.
26	<code>if hashlib.sha256(data).hexdigest() != 'b5d2e486645cb48571b30d86a0ffcc343c33a5a797a71dbd32fc025f5af66d7c':</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é obter a biblioteca do tutorial e conferir sua integridade.
27	<code>raise RuntimeError('O pacote oficial mudou. Conferir nova versao antes de atualizar a dependencia.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
28	<code>with zipfile.ZipFile(io.BytesIO(data)) as archive:</code>	Abre um recurso dentro de um bloco que cuida de fechá-lo ao terminar, inclusive se houver erro.
29	<code>candidates = [name for name in archive.namelist() if name.endswith('/lib/search.jar') and '__MACOSX' not in name]</code>	Define ou atualiza candidates com a expressão à direita. Essa informação participa da rotina de obter a biblioteca do tutorial e conferir sua integridade.
30	<code>if len(candidates)!=1: raise RuntimeError('search.jar nao localizado de forma univoca no pacote.')</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é obter a biblioteca do tutorial e conferir sua integridade.
31	<code>binary = archive.read(candidates[0])</code>	Define ou atualiza binary com a expressão à direita. Essa informação participa da rotina de obter a biblioteca do tutorial e conferir sua integridade.
32	<code>if hashlib.sha256(binary).hexdigest()!=expected: raise RuntimeError('Integridade de search.jar divergente.')</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é obter a biblioteca do tutorial e conferir sua integridade.
33	<code>target.parent.mkdir(exist_ok=True)</code>	Cria a pasta de trabalho ou de registros antes de gravar os arquivos.
34	<code>target.write_bytes(binary)</code>	Grava o conteúdo no arquivo indicado. O caminho e a codificação definem onde o registro ficará disponível.
35	<code>if hashlib.sha256(target.read_bytes()).hexdigest()!= expected:</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é obter a biblioteca do tutorial e conferir sua integridade.

Linha	Código	Como ler
36	<pre>raise RuntimeError('search.jar local difere da dependencia verificada.')</pre>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
38	<pre>def environment():</pre>	Define a função para selecionar o Java e as variáveis somente para o processo do laboratório. Os parâmetros são os dados que a chamada recebe.
39	<pre>env = os.environ.copy()</pre>	Define ou atualiza env com a expressão à direita. Essa informação participa da rotina de selecionar o Java e as variáveis somente para o processo do laboratório.
40	<pre>local = ROOT.parent / 'JaCaMo/00-ambiente-e-instalacao'</pre>	Define ou atualiza local com a expressão à direita. Essa informação participa da rotina de selecionar o Java e as variáveis somente para o processo do laboratório.
41	<pre>if (local/'jdk-21/bin/java.exe').exists():</pre>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é selecionar o Java e as variáveis somente para o processo do laboratório.
42	<pre>env['JAVA_HOME'] = str(local/'jdk-21')</pre>	Define ou atualiza env['JAVA_HOME'] com a expressão à direita. Essa informação participa da rotina de selecionar o Java e as variáveis somente para o processo do laboratório.
43	<pre>env['GRADLE_USER_HOME'] = str(local/'gradle-user-home')</pre>	Define ou atualiza env['GRADLE_USER_HOME'] com a expressão à direita. Essa informação participa da rotina de selecionar o Java e as variáveis somente para o processo do laboratório.
44	<pre>env['GRADLE_OPTS'] = '-Duser.home="'+str(local/'perfil-isolado')+''''</pre>	Define ou atualiza env['GRADLE_OPTS'] com a expressão à direita. Essa informação participa da rotina de selecionar o Java e as variáveis somente para o processo do laboratório.
45	<pre>env.pop('JASON_HOME',None)</pre>	Retira JASON_HOME da cópia das variáveis do processo para evitar seleção de uma instalação externa.
46	<pre>binary = 'java.exe' if os.name=='nt' else 'java'</pre>	Define ou atualiza binary com a expressão à direita. Essa informação participa da rotina de selecionar o Java e as variáveis somente para o processo do laboratório.
47	<pre>candidate = Path(env.get('JAVA_HOME',''))/'bin'/binary</pre>	Define ou atualiza candidate com a expressão à direita. Essa informação participa da rotina de selecionar o Java e as variáveis somente para o processo do laboratório.
48	<pre>java = str(candidate) if candidate.is_file() else shutil.which(binary)</pre>	Define ou atualiza java com a expressão à direita. Essa informação participa da rotina de selecionar o Java e as variáveis somente para o processo do laboratório.
49	<pre>if not java:</pre>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é selecionar o Java e as variáveis somente para o processo do laboratório.
50	<pre>raise RuntimeError('Java 21 nao encontrado. Use o laboratorio JaCaMo ou configure JAVA_HOME para o processo.')</pre>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
51	<pre>return java,env</pre>	Entrega este resultado a quem chamou a rotina de selecionar o Java e as variáveis somente para o processo do laboratório.
53	<pre>def compile_project():</pre>	Define a função para compilar as fontes e registrar o classpath. Os parâmetros são os dados que a chamada recebe.
54	<pre>prepare_search_library()</pre>	Executa a chamada indicada como parte da rotina de compilar as fontes e registrar o classpath. Seus argumentos indicam os dados usados nesta etapa.
55	<pre>_,env = environment()</pre>	Define ou atualiza _env com a expressão à direita. Essa informação participa da rotina de compilar as fontes e registrar o classpath.

Linha	Código	Como ler
56	<code>wrapper = ROOT/('gradlew.bat' if os.name=='nt' else 'gradlew')</code>	Define ou atualiza wrapper com a expressão à direita. Essa informação participa da rotina de compilar as fontes e registrar o classpath.
57	<code>command = [str(wrapper), '--no-daemon', '--console=plain', 'classes', 'testClasses', 'runtimeClasspath']</code>	Define ou atualiza command com a expressão à direita. Essa informação participa da rotina de compilar as fontes e registrar o classpath.
58	<code>if os.name!='nt': command.insert(0, 'sh')</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é compilar as fontes e registrar o classpath.
59	<code>subprocess.run(command, cwd=ROOT, env=env, check=True, timeout=180)</code>	Executa ou configura um processo externo. Os argumentos, ambiente, captura de saída e prazo aparecem nesta chamada.
61	<code>def folder(kind):</code>	Define a função para criar uma pasta exclusiva para os registros da execução. Os parâmetros são os dados que a chamada recebe.
62	<code>stamp = datetime.now().strftime('%Y%m%d-%H%M%S-%f')</code>	Define ou atualiza stamp com a expressão à direita. Essa informação participa da rotina de criar uma pasta exclusiva para os registros da execução.
63	<code>path = ROOT/'experimentos/execucoes'/f'{stamp}-{kind}'</code>	Define ou atualiza path com a expressão à direita. Essa informação participa da rotina de criar uma pasta exclusiva para os registros da execução.
64	<code>path.mkdir(parents=True)</code>	Cria a pasta de trabalho ou de registros antes de gravar os arquivos.
65	<code>return path</code>	Entrega este resultado a quem chamou a rotina de criar uma pasta exclusiva para os registros da execução.
67	<code>def java_command(main, args, test=False, gui=False):</code>	Define a função para montar os argumentos Java e o perfil local de execução. Os parâmetros são os dados que a chamada recebe.
68	<code>java, env = environment()</code>	Define ou atualiza java, env com a expressão à direita. Essa informação participa da rotina de montar os argumentos Java e o perfil local de execução.
69	<code>runtime_home = ROOT/'build/perfil-java'</code>	Define ou atualiza runtime_home com a expressão à direita. Essa informação participa da rotina de montar os argumentos Java e o perfil local de execução.
70	<code>runtime_home.mkdir(parents=True, exist_ok=True)</code>	Cria a pasta de trabalho ou de registros antes de gravar os arquivos.
71	<code>cpfile = ROOT/'build/' + ('test-runtime-classpath.txt' if test else 'runtime-classpath.txt')</code>	Define ou atualiza cpfile com a expressão à direita. Essa informação participa da rotina de montar os argumentos Java e o perfil local de execução.
72	<code>if not cpfile.exists(): compile_project()</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é montar os argumentos Java e o perfil local de execução.
73	<code>command = [java, '-Dfile.encoding=UTF-8', '-Dstdout.encoding=UTF-8', '-Dstderr.encoding=UTF-8', '-Dgoldminers.headless='+str(not gui).lower(),</code>	Define ou atualiza command com a expressão à direita. Essa informação participa da rotina de montar os argumentos Java e o perfil local de execução.
74	<code>'-Duser.home='+str(runtime_home), '-Djava.util.logging.config.file=logging.properties',</code>	Define ou atualiza '-Duser.home com a expressão à direita. Essa informação participa da rotina de montar os argumentos Java e o perfil local de execução.
75	<code>'-Djava.awt.headless='+str(not gui).lower(), '-Dgoldminers.delay=20',</code>	Define ou atualiza '-Djava.awt.headless com a expressão à direita. Essa informação participa da rotina de montar os argumentos Java e o perfil local de execução.

Linha	Código	Como ler
76	<pre>'_ cp',cpfile.read_text(encoding='utf-8'),main,*args]</pre>	Lê o conteúdo do arquivo indicado para conferir, calcular ou preparar a próxima etapa.
77	<pre>return command,env</pre>	Entrega este resultado a quem chamou a rotina de montar os argumentos Java e o perfil local de execução.
79	<pre>def run_java(main,args,output,test=False,seconds=45,g ui=False):</pre>	Define a função para iniciar o processo e encerrá-lo ao alcançar o limite de tempo. Os parâmetros são os dados que a chamada recebe.
80	<pre>command,env = java_command(main,args,test,gui)</pre>	Define ou atualiza command,env com a expressão à direita. Essa informação participa da rotina de iniciar o processo e encerrá-lo ao alcançar o limite de tempo.
81	<pre>with (output/'stdout.log').open('w',encoding='utf-8') as out, (output/'stderr.log').open('w',encoding='utf-8') as err:</pre>	Abre um recurso dentro de um bloco que cuida de fechá-lo ao terminar, inclusive se houver erro.
82	<pre>process = subprocess.Popen(command,cwd=ROOT,env=env,stdout= out,stderr=err,</pre>	Executa ou configura um processo externo. Os argumentos, ambiente, captura de saída e prazo aparecem nesta chamada.
83	<pre>creationflags=subprocess.CREATE_NO_WINDOW if os.name=='nt' and not gui else 0)</pre>	Executa ou configura um processo externo. Os argumentos, ambiente, captura de saída e prazo aparecem nesta chamada.
84	<pre>expired = False</pre>	Começa assumindo que o processo ainda não atingiu o limite de tempo.
85	<pre>try: code = process.wait(timeout=seconds)</pre>	Tenta concluir esta chamada dentro do tratamento de erros que vem a seguir; wait acompanha o processo até terminar ou atingir o prazo.
86	<pre>except subprocess.TimeoutExpired:</pre>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
87	<pre>expired = True</pre>	Registra que o encerramento foi provocado pelo prazo máximo da sessão.
88	<pre>process.terminate()</pre>	Pede o encerramento apenas do processo Java iniciado por esta chamada.
89	<pre>try: code = process.wait(timeout=8)</pre>	Tenta concluir esta chamada dentro do tratamento de erros que vem a seguir; wait acompanha o processo até terminar ou atingir o prazo.
90	<pre>except subprocess.TimeoutExpired:</pre>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
91	<pre>process.kill()); code = process.wait(timeout=5)</pre>	Força o fim do processo iniciado se ele não terminar e recolhe seu código de saída.
92	<pre>return code,expired, (output/'stdout.log').read_text(encoding='utf-8')+ (output/'stderr.log').read_text(encoding='utf-8')</pre>	Entrega este resultado a quem chamou a rotina de iniciar o processo e encerrá-lo ao alcançar o limite de tempo.
94	<pre>def validate():</pre>	Define a função para conferir o modelo e uma entrega, preservando evidências. Os parâmetros são os dados que a chamada recebe.
95	<pre>compile_project()</pre>	Executa a chamada indicada como parte da rotina de conferir o modelo e uma entrega, preservando evidências. Seus argumentos indicam os dados usados nesta etapa.

Linha	Código	Como ler
96	<code>destination = folder('verificacao')</code>	Define ou atualiza destination com a expressão à direita. Essa informação participa da rotina de conferir o modelo e uma entrega, preservando evidências.
97	<code>model = destination/'modelo'; model.mkdir()</code>	Cria a pasta de trabalho ou de registros antes de gravar os arquivos.
98	<code>code,expired,text = run_java('mining.ModelChecks',[],model,test=True)</code>	Define ou atualiza code,expired,text com a expressão à direita. Essa informação participa da rotina de conferir o modelo e uma entrega, preservando evidências.
99	<code>if code!=0 or expired or 'MODEL_CHECKS passed=10' not in text:</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é conferir o modelo e uma entrega, preservando evidências.
100	<code>raise RuntimeError('Falha no modelo. Veja '+str(model))</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
101	<code>integration = destination/'integracao'; integration.mkdir()</code>	Cria a pasta de trabalho ou de registros antes de gravar os arquivos.
102	<code>code,expired,text = run_java('jacamo.infra.JaCaMoLauncher', ['configuracoes/verificacao.jcm'],integration,test=True)</code>	Define ou atualiza code,expired,text com a expressão à direita. Essa informação participa da rotina de conferir o modelo e uma entrega, preservando evidências.
103	<code>required = ['PERCEPTION gold=1,1','PICK carrying=true','WORLD_VERIFIED deposited=1 carrying=false conservation=ok','END status=ok']</code>	Define ou atualiza required com a expressão à direita. Essa informação participa da rotina de conferir o modelo e uma entrega, preservando evidências.
104	<code>missing = [marker for marker in required if marker not in text]</code>	Define ou atualiza missing com a expressão à direita. Essa informação participa da rotina de conferir o modelo e uma entrega, preservando evidências.
105	<code>if code!=0 or expired or missing or 'GOLD_MINERS FAIL' in text or re.search(r'Exception SEVERE Error parsing Error creating',text):</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é conferir o modelo e uma entrega, preservando evidências.
106	<code>raise RuntimeError('Falha de integracao. Marcadores ausentes: '+str(missing)+''. Veja '+str(integration))</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
107	<code>source_files = sorted(list((ROOT/'src').rglob('*.java'))+list((ROOT/'src').rglob('*.asl'))+[ROOT/'build.gradle',ROOT/'configuracoes/verificacao.jcm',ROOT/'logging.properties',Path(__file__)])</code>	Define ou atualiza source_files com a expressão à direita. Essa informação participa da rotina de conferir o modelo e uma entrega, preservando evidências.
108	<code>result = {'data_utc':datetime.now(timezone.utc).isoformat(),'status':'aprovado','verificacoes_modelo':10,</code>	Define ou atualiza result com a expressão à direita. Essa informação participa da rotina de conferir o modelo e uma entrega, preservando evidências.
109	<code>'integracao':'Jason e CArtaG0 executaram percepcao, movimento, coleta e deposito; estado conferido por observador de teste.'</code>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.
110	<code>'limite':'Nao e campeonato, comparacao de estrategias nem implementacao MAPC 2006 completa.'</code>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.

Linha	Código	Como ler
111	<pre>'marcadores':required,'fontes_sha256': {str(p.relative_to(ROOT)).replace('\\','/'):hashl ib.sha256(p.read_bytes()).hexdigest() for p in source_files}}</pre>	Calcula o resumo SHA-256 para identificar exatamente os bytes verificados, sem interpretar o conteúdo como código.
112	<pre>(destination/'resultado.json').write_text(json.du mps(result,ensure_ascii=False,indent=2),encoding= 'utf-8')</pre>	Converte a estrutura de dados para JSON legível, preservando a identificação da origem dos números.
113	<pre>evidence = ROOT/'experimentos/evidencias'</pre>	Define ou atualiza evidence com a expressão à direita. Essa informação participa da rotina de conferir o modelo e uma entrega, preservando evidências.
114	<pre>(evidence/'verificacao- inicial.json').write_text(json.dumps(result,ensur e_ascii=False,indent=2),encoding='utf-8')</pre>	Converte a estrutura de dados para JSON legível, preservando a identificação da origem dos números.
115	<pre>print('VERIFICACAO_GOLD_MINERS_APROVADA modelo=10 integracao=1')</pre>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
116	<pre>print(destination)</pre>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
118	<pre>def execute(args):</pre>	Define a função para gerar a configuração de uma sessão de estudo e acompanhar sua saída. Os parâmetros são os dados que a chamada recebe.
119	<pre>agent = 'miner.asl' if args.modos=='explorar' else 'miner-coleta.asl'</pre>	Define ou atualiza agent com a expressão à direita. Essa informação participa da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída.
120	<pre>if args.modos=='coletar' and args.cenario not in (1,2,3):</pre>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é gerar a configuração de uma sessão de estudo e acompanhar sua saída.
121	<pre>raise ValueError('A base de coleta usa deposito 0,0. Use cenários 1 a 3 antes de adaptar o agente.')</pre>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
122	<pre>compile_project()</pre>	Executa a chamada indicada como parte da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída. Seus argumentos indicam os dados usados nesta etapa.
123	<pre>destination = folder('tutorial')</pre>	Define ou atualiza destination com a expressão à direita. Essa informação participa da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída.
124	<pre>generated = ROOT/'configuracoes/geradas'; generated.mkdir(exist_ok=True)</pre>	Cria a pasta de trabalho ou de registros antes de gravar os arquivos.
125	<pre>jcm = generated/(destination.name+'.jcm')</pre>	Define ou atualiza jcm com a expressão à direita. Essa informação participa da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída.
126	<pre>agents = ['agent leader']</pre>	Define ou atualiza agents com a expressão à direita. Essa informação participa da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída.
127	<pre>for i in range(1,5):</pre>	Percorre os elementos ou índices indicados, repetindo as operações do corpo para cada um.
128	<pre>source = agent if args.modos=='coletar' or i==1 else 'dummy.asl'</pre>	Define ou atualiza source com a expressão à direita. Essa informação participa da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída.

Linha	Código	Como ler
129	<pre>agents.append(f'agent miner{i}: {source} {{\n focus: mining.m{i}view\n}}')</pre>	Executa a chamada indicada como parte da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída. Seus argumentos indicam os dados usados nesta etapa.
130	<pre>artifacts = [f'artifact m{i}view: mining.MiningPlanet({args.cenario},{i-1})' for i in range(1,5)]</pre>	Define ou atualiza artifacts com a expressão à direita. Essa informação participa da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída.
131	<pre>jcm.write_text('mas tutorial_gold_miners {\n'+'\n'.join(agents)+'\nworkspace mining {\n'+'\n'.join(artifacts)+'\n}\n',encoding='utf-8')</pre>	Grava o conteúdo no arquivo indicado. O caminho e a codificação definem onde o registro ficará disponível.
132	<pre>shutil.copy2(jcm,destination/'cenario.jcm')</pre>	Preserva uma cópia do arquivo indicado, vinculando a configuração à execução.
133	<pre>code,expired,text = run_java('jacamo.infra.JaCaMoLauncher', [jcm.relative_to(ROOT).as_posix()],destination,se conds=args.segundos,gui=args.interface)</pre>	Define ou atualiza code,expired,text com a expressão à direita. Essa informação participa da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída.
134	<pre>failures = bool(re.search(r'Exception SEVERE parse error Error parsing Error creating',text))</pre>	Define ou atualiza failures com a expressão à direita. Essa informação participa da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída.
135	<pre>result = {'modo':args.modo,'cenario':args.cenario,'duracao _limite_segundos':args.segundos,</pre>	Define ou atualiza result com a expressão à direita. Essa informação participa da rotina de gerar a configuração de uma sessão de estudo e acompanhar sua saída.
136	<pre>'encerrado_pelo_limite':expired,'codigo_processo' :code,'erro_detectado':failures,</pre>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.
137	<pre>'observacao':'Sessao de estudo; encerramento por tempo nao significa que todo o ouro foi coletado.'}</pre>	Continuação da expressão ou estrutura iniciada acima. Leia-a junto à linha anterior para acompanhar a operação completa.
138	<pre>(destination/'sessao.json').write_text(json.dumps (result,ensure_ascii=False,indent=2),encoding='utf-8')</pre>	Converte a estrutura de dados para JSON legível, preservando a identificação da origem dos números.
139	<pre>print(json.dumps(result,ensure_ascii=False,indent =2)); print(destination)</pre>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
140	<pre>if failures or (not expired and code!=0): raise RuntimeError('A sessao apresentou falha. Confira os registros.')</pre>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é gerar a configuração de uma sessão de estudo e acompanhar sua saída.
142	<pre>def main():</pre>	Define a função para interpretar o comando informado e encaminhá-lo à operação correspondente. Os parâmetros são os dados que a chamada recebe.
143	<pre>parser = argparse.ArgumentParser(description=__doc__)</pre>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.
144	<pre>sub = parser.add_subparsers(dest='command',required=True)</pre>	Define ou atualiza sub com a expressão à direita. Essa informação participa da rotina de interpretar o comando informado e encaminhá-lo à operação correspondente.
145	<pre>sub.add_parser('diagnosticar'); sub.add_parser('compilar'); sub.add_parser('validar')</pre>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.
146	<pre>p = sub.add_parser('executar')</pre>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.

Linha	Código	Como ler
147	<code>p.add_argument('--modo',choices=['explorar','coletar'],default='coletar')</code>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.
148	<code>p.add_argument('--cenario',type=int,choices=range(1,7),default=3)</code>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.
149	<code>p.add_argument('--segundos',type=int,default=15)</code>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.
150	<code>p.add_argument('--interface',action='store_true',help='Abre a visualizacao grafica para estudo interativo.')</code>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.
151	<code>args = parser.parse_args()</code>	Lê os argumentos do terminal conforme as opções declaradas acima.
152	<code>if args.command=='diagnosticar':</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é interpretar o comando informado e encaminhá-lo à operação correspondente.
153	<code>java,env = environment()</code>	Define ou atualiza java,env com a expressão à direita. Essa informação participa da rotina de interpretar o comando informado e encaminhá-lo à operação correspondente.
154	<code>print('Java:',java); print('JaCaMo: 1.3.0 Gradle: 8.10')</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.
155	<code>subprocess.run([java,'-version'],env=env,check=True)</code>	Executa ou configura um processo externo. Os argumentos, ambiente, captura de saída e prazo aparecem nesta chamada.
156	<code>elif args.command=='compilar': compile_project()</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é interpretar o comando informado e encaminhá-lo à operação correspondente.
157	<code>elif args.command=='validar': validate()</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é interpretar o comando informado e encaminhá-lo à operação correspondente.
158	<code>else:</code>	Trata a alternativa em que a condição anterior não foi satisfeita.
159	<code>if not 1<=args.segundos<=3600: parser.error('Use duracao de 1 a 3600 segundos.')</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é interpretar o comando informado e encaminhá-lo à operação correspondente.
160	<code>execute(args)</code>	Executa a chamada indicada como parte da rotina de interpretar o comando informado e encaminhá-lo à operação correspondente. Seus argumentos indicam os dados usados nesta etapa.
162	<code>if __name__=='__main__': main()</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é interpretar o comando informado e encaminhá-lo à operação correspondente.

scripts/indicadores.py

Calcula eficiência, eficácia e diferença de resultado útil a partir de registros informados. Conserva a identificação de números ilustrativos.

[Abrir arquivo original](#) · 87 linhas · numeração da cópia documentada

Linha	Código	Como ler
2	<code>from pathlib import Path</code>	Trabalha com caminhos e arquivos sem montar separadores manualmente.

Linha	Código	Como ler
3	<code>import argparse</code>	Lê comandos e opções digitados no terminal, com ajuda e validação de argumentos.
4	<code>import json</code>	Lê e grava configurações e resultados em JSON.
5	<code>import math</code>	Permite rejeitar medidas que sejam infinitas ou indefinidas.
7	<code>def numero(dados, nome):</code>	Define a função para rejeitar valores que não servem como medidas não negativas. Os parâmetros são os dados que a chamada recebe.
8	<code>valor = dados[nome]</code>	Define ou atualiza valor com a expressão à direita. Essa informação participa da rotina de rejeitar valores que não servem como medidas não negativas.
9	<code>if isinstance(valor, bool) or not isinstance(valor, (int, float)):</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é rejeitar valores que não servem como medidas não negativas.
10	<code>raise ValueError(f'{nome} precisa ser numerico.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
11	<code>if not math.isfinite(valor) or valor < 0:</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é rejeitar valores que não servem como medidas não negativas.
12	<code>raise ValueError(f'{nome} precisa ser finito e nao negativo.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
13	<code>return valor</code>	Entrega este resultado a quem chamou a rotina de rejeitar valores que não servem como medidas não negativas.
15	<code>def avaliar(registro):</code>	Define a função para calcular os indicadores de uma equipe em um registro. Os parâmetros são os dados que a chamada recebe.
17	<code>valor = numero(registro, 'valor_confirmado')</code>	Define ou atualiza valor com a expressão à direita. Essa informação participa da rotina de calcular os indicadores de uma equipe em um registro.
18	<code>acoes = numero(registro, 'acoes_consumidas')</code>	Define ou atualiza acoes com a expressão à direita. Essa informação participa da rotina de calcular os indicadores de uma equipe em um registro.
19	<code>meta = numero(registro, 'meta_valor')</code>	Define ou atualiza meta com a expressão à direita. Essa informação participa da rotina de calcular os indicadores de uma equipe em um registro.
20	<code>passo = numero(registro, 'passo_final')</code>	Define ou atualiza passo com a expressão à direita. Essa informação participa da rotina de calcular os indicadores de uma equipe em um registro.
21	<code>limite = numero(registro, 'limite_passos')</code>	Define ou atualiza limite com a expressão à direita. Essa informação participa da rotina de calcular os indicadores de uma equipe em um registro.
22	<code>if meta == 0 or limite == 0:</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é calcular os indicadores de uma equipe em um registro.
23	<code>raise ValueError('Meta e limite de passos precisam ser maiores que zero.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
24	<code>if any(v != int(v) for v in (acoes, passo, limite)):</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é calcular os indicadores de uma equipe em um registro.
25	<code>raise ValueError('Acoes e passos precisam ser inteiros.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.

Linha	Código	Como ler
26	<code>if valor > 0 and acoes == 0:</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é calcular os indicadores de uma equipe em um registro.
27	<code>raise ValueError('Entrega positiva com zero acoes nao e um registro valido deste protocolo.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
28	<code>dentro_prazo = passo <= limite</code>	Define ou atualiza <code>dentro_prazo</code> com a expressão à direita. Essa informação participa da rotina de calcular os indicadores de uma equipe em um registro.
29	<code>return {</code>	Entrega este resultado a quem chamou a rotina de calcular os indicadores de uma equipe em um registro.
30	<code>'eficiencia_valor_por_acao': valor / acoes if acoes else None,</code>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.
31	<code>'eficacia_fracao_da_meta': min(valor / meta, 1) if dentro_prazo else 0,</code>	Executa a chamada indicada como parte da rotina de calcular os indicadores de uma equipe em um registro. Seus argumentos indicam os dados usados nesta etapa.
32	<code>'meta_atingida_no_prazo': valor >= meta and dentro_prazo,</code>	Define ou atualiza <code>'meta_atingida_no_prazo': valor ></code> com a expressão à direita. Essa informação participa da rotina de calcular os indicadores de uma equipe em um registro.
33	<code>'valor_confirmado': valor,</code>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.
34	<code>'limite': 'Acoes devem incluir falhas e espera; custo de mensagens e CPU e relatado separadamente.'</code>	Continuação da expressão ou estrutura iniciada acima. Leia-a junto à linha anterior para acompanhar a operação completa.
35	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
37	<code>def comparar(dados):</code>	Define a função para conferir os pares e calcular diferenças entre referência e candidata. Os parâmetros são os dados que a chamada recebe.
39	<code>origem = dados['origem']</code>	Define ou atualiza <code>origem</code> com a expressão à direita. Essa informação participa da rotina de conferir os pares e calcular diferenças entre referência e candidata.
40	<code>if origem not in ('ilustrativo', 'observado'):</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é conferir os pares e calcular diferenças entre referência e candidata.
41	<code>raise ValueError('Informe origem ilustrativo ou observado.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
42	<code>pares = dados['pares']</code>	Define ou atualiza <code>pares</code> com a expressão à direita. Essa informação participa da rotina de conferir os pares e calcular diferenças entre referência e candidata.
43	<code>if not pares:</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é conferir os pares e calcular diferenças entre referência e candidata.
44	<code>raise ValueError('Informe pelo menos um par de registros.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
45	<code>resultados = []</code>	Define ou atualiza <code>resultados</code> com a expressão à direita. Essa informação participa da rotina de conferir os pares e calcular diferenças entre referência e candidata.

Linha	Código	Como ler
46	<code>for par in pares:</code>	Percorre os elementos ou índices indicados, repetindo as operações do corpo para cada um.
47	<code>referencia = par['referencia']</code>	Define ou atualiza referencia com a expressão à direita. Essa informação participa da rotina de conferir os pares e calcular diferenças entre referência e candidata.
48	<code>candidata = par['candidata']</code>	Define ou atualiza candidata com a expressão à direita. Essa informação participa da rotina de conferir os pares e calcular diferenças entre referência e candidata.
49	<code>for chave in ('contexto', 'semente', 'orientacao', 'meta_valor', 'limite_passos'):</code>	Percorre os elementos ou índices indicados, repetindo as operações do corpo para cada um.
50	<code>if referencia[chave] != candidata[chave]:</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é conferir os pares e calcular diferenças entre referência e candidata.
51	<code>raise ValueError(f'Par {par["id"]} nao e comparavel no campo {chave}.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
52	<code>base = avaliar(referencia)</code>	Define ou atualiza base com a expressão à direita. Essa informação participa da rotina de conferir os pares e calcular diferenças entre referência e candidata.
53	<code>nova = avaliar(candidata)</code>	Define ou atualiza nova com a expressão à direita. Essa informação participa da rotina de conferir os pares e calcular diferenças entre referência e candidata.
54	<code>ganho = nova['valor_confirmado'] - base['valor_confirmado']</code>	Define ou atualiza ganho com a expressão à direita. Essa informação participa da rotina de conferir os pares e calcular diferenças entre referência e candidata.
55	<code>resultados.append({</code>	Executa a chamada indicada como parte da rotina de conferir os pares e calcular diferenças entre referência e candidata. Seus argumentos indicam os dados usados nesta etapa.
56	<code>'id': par['id'], 'referencia': base, 'candidata': nova,</code>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.
57	<code>'diferenca_valor_confirmado': ganho,</code>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.
58	<code>'variacao_percentual': 100 * ganho / base['valor_confirmado'] if base['valor_confirmado'] else None</code>	Continuação da expressão ou estrutura iniciada acima. Leia-a junto à linha anterior para acompanhar a operação completa.
59	<code>})</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
60	<code>return {</code>	Entrega este resultado a quem chamou a rotina de conferir os pares e calcular diferenças entre referência e candidata.
61	<code>'origem': origem,</code>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.
62	<code>'pares': resultados,</code>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.
63	<code>'efetividade_simulada': {</code>	Continuação da expressão ou estrutura iniciada acima. Leia-a junto à linha anterior para acompanhar a operação completa.
64	<code>'diferenca_media_valor_confirmado': sum(p['diferenca_valor_confirmado'] for p in resultados) / len(resultados),</code>	Executa a chamada indicada como parte da rotina de conferir os pares e calcular diferenças entre referência e candidata. Seus argumentos indicam os dados usados nesta etapa.

Linha	Código	Como ler
65	<code>'pares_com_melhora': sum(p['diferenca_valor_confirmado'] > 0 for p in resultados),</code>	Executa a chamada indicada como parte da rotina de conferir os pares e calcular diferenças entre referência e candidata. Seus argumentos indicam os dados usados nesta etapa.
66	<code>'total_pares': len(resultados),</code>	Executa a chamada indicada como parte da rotina de conferir os pares e calcular diferenças entre referência e candidata. Seus argumentos indicam os dados usados nesta etapa.
67	<code>'interpretacao': 'Efeito util candidato no cenario declarado; depende de desenho comparavel e evidencia de entrega.',</code>	Continua a estrutura de dados ou de parâmetros iniciada acima; o significado do valor é dado pela chave correspondente.
68	<code>'limite': 'Nao demonstra impacto profissional, significancia estatistica ou causalidade apenas pelo calculo.'</code>	Continuação da expressão ou estrutura iniciada acima. Leia-a junto à linha anterior para acompanhar a operação completa.
69	<code>},</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
70	<code>'status': 'EXEMPLO_DIDATICO_SEM_RESULTADO_EXPERIMENTAL' if origem == 'ilustrativo' else 'REGISTROS_INFORMADOS_PARA_ANALISE'</code>	Define ou atualiza 'status': 'EXEMPLO_DIDATICO_SEM_RESULTADO_EXPERIMENTAL' if origem com a expressão à direita. Essa informação participa da rotina de conferir os pares e calcular diferenças entre referência e candidata.
71	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
73	<code>def main():</code>	Define a função para interpretar o comando informado e encaminhá-lo à operação correspondente. Os parâmetros são os dados que a chamada recebe.
74	<code>parser = argparse.ArgumentParser(description=__doc__)</code>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.
75	<code>parser.add_argument('entrada', type=Path)</code>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.
76	<code>parser.add_argument('--saida', type=Path)</code>	Define um comando ou uma opção do terminal, com tipo, padrão ou valores aceitos.
77	<code>args = parser.parse_args()</code>	Lê os argumentos do terminal conforme as opções declaradas acima.
78	<code>dados = json.loads(args.entrada.read_text(encoding='utf- 8'))</code>	Lê a estrutura JSON de entrada para trabalhar com os seus campos.
79	<code>resultado = comparar(dados)</code>	Define ou atualiza resultado com a expressão à direita. Essa informação participa da rotina de interpretar o comando informado e encaminhá-lo à operação correspondente.
80	<code>texto = json.dumps(resultado, ensure_ascii=False, indent=2)</code>	Converte a estrutura de dados para JSON legível, preservando a identificação da origem dos números.
81	<code>if args.saida:</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é interpretar o comando informado e encaminhá-lo à operação correspondente.
82	<code>args.saida.parent.mkdir(parents=True, exist_ok=True)</code>	Cria a pasta de trabalho ou de registros antes de gravar os arquivos.
83	<code>args.saida.write_text(texto, encoding='utf-8')</code>	Grava o conteúdo no arquivo indicado. O caminho e a codificação definem onde o registro ficará disponível.
84	<code>print(texto)</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.

Linha	Código	Como ler
86	<code>if __name__ == '__main__':</code>	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é interpretar o comando informado e encaminhá-lo à operação correspondente.
87	<code>main()</code>	Executa a chamada indicada como parte da rotina de interpretar o comando informado e encaminhá-lo à operação correspondente. Seus argumentos indicam os dados usados nesta etapa.

scripts/verificar_indicadores.py

Verifica fórmulas e entradas problemáticas. Inclui zero ações, prazo excedido, números inválidos e pares não comparáveis.

[Abrir arquivo original](#) · 38 linhas · numeração da cópia documentada

Linha	Código	Como ler
2	<code>import copy</code>	Cria cópias independentes dos exemplos para testar entradas inválidas sem alterar a referência.
3	<code>import json</code>	Lê e grava configurações e resultados em JSON.
4	<code>from pathlib import Path</code>	Trabalha com caminhos e arquivos sem montar separadores manualmente.
5	<code>from indicadores import avaliar, comparar</code>	Reutiliza os cálculos do módulo de indicadores para conferir seus resultados e casos de erro.
7	<code>root = Path(__file__).resolve().parents[1]</code>	Define ou atualiza root com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
8	<code>dados = json.loads((root/'experimentos/exemplo-indicadores.json').read_text(encoding='utf-8'))</code>	Lê a estrutura JSON de entrada para trabalhar com os seus campos.
9	<code>resultado = comparar(dados)</code>	Define ou atualiza resultado com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
10	<code>assert abs(resultado['pares'][0]['candidata'] ['eficiencia_valor_por_acao'] - 26/90) < 1e-12</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
11	<code>assert resultado['pares'][0]['candidata'] ['meta_atingida_no_prazo'] is True</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
12	<code>assert resultado['pares'][1]['candidata'] ['meta_atingida_no_prazo'] is False</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
13	<code>assert resultado['efetividade_simulada'] ['diferenca_media_valor_confirmado'] == 6.5</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
14	<code>assert resultado['origem'] == 'ilustrativo'</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
15	<code>zero = copy.deepcopy(dados['pares'][0] ['referencia'])</code>	Define ou atualiza zero com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
16	<code>zero.update(valor_confirmado=0, acoes_consumidas=0)</code>	Define ou atualiza zero.update(valor_confirmado com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.

Linha	Código	Como ler
17	<code>assert avaliar(zero)['eficiencia_valor_por_acao'] is None</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
18	<code>tardio = copy.deepcopy(dados['pares'][0]['candidata'])</code>	Define ou atualiza tardio com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
19	<code>tardio['passo_final'] = 101</code>	Define ou atualiza tardio['passo_final'] com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
20	<code>assert avaliar(tardio)['meta_atingida_no_prazo'] is False</code>	Exige que a condição descrita na expressão seja verdadeira. A falha interrompe a verificação e indica o que precisa ser revisto.
21	<code>for campo,valor in [('acoes_consumidas',-1), ('valor_confirmado',float('nan')), ('meta_valor',0),('acoes_consumidas',0)]:</code>	Percorre os elementos ou índices indicados, repetindo as operações do corpo para cada um.
22	<code>invalido = copy.deepcopy(dados['pares'][0]['candidata'])</code>	Define ou atualiza invalido com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
23	<code>invalido[campo] = valor</code>	Define ou atualiza invalido[campo] com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
24	<code>try:</code>	Tenta concluir esta chamada dentro do tratamento de erros que vem a seguir; wait acompanha o processo até terminar ou atingir o prazo.
25	<code>avaliar(invalido)</code>	Executa a chamada indicada como parte da rotina de preparação deste arquivo. Seus argumentos indicam os dados usados nesta etapa.
26	<code>except ValueError:</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
27	<code>pass</code>	Neste ramo Python não é necessário fazer outra operação; a exceção esperada já foi reconhecida.
28	<code>else:</code>	Trata a alternativa em que a condição anterior não foi satisfeita.
29	<code>raise AssertionError('Registro invalido aceito: '+campo)</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
30	<code>desigual = copy.deepcopy(dados)</code>	Define ou atualiza desigual com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
31	<code>desigual['pares'][0]['candidata']['semente'] = 999</code>	Define ou atualiza desigual['pares'][0]['candidata']['semente'] com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.
32	<code>try:</code>	Tenta concluir esta chamada dentro do tratamento de erros que vem a seguir; wait acompanha o processo até terminar ou atingir o prazo.
33	<code>comparar(desigual)</code>	Executa a chamada indicada como parte da rotina de preparação deste arquivo. Seus argumentos indicam os dados usados nesta etapa.
34	<code>except ValueError:</code>	Trata o erro indicado. O corpo abaixo define se ele será registrado, convertido em falha ou propagado.
35	<code>pass</code>	Neste ramo Python não é necessário fazer outra operação; a exceção esperada já foi reconhecida.
36	<code>else:</code>	Trata a alternativa em que a condição anterior não foi satisfeita.

Linha	Código	Como ler
37	<code>raise AssertionError('Par com sementes distintas foi aceito.')</code>	Interrompe este caminho com um erro explícito, evitando apresentar a condição inválida como sucesso.
38	<code>print('INDICADORES_APROVADOS calculos, prazo, origem, zero e entradas invalidas conferidos')</code>	Escreve uma informação no registro de execução. No teste, os marcadores são lidos pelo lançador para conferir o percurso.

build.gradle

Declara ferramentas, dependências, pastas de código e tarefas Java. O lançador obtém search.jar antes de chamar o Gradle.

[Abrir arquivo original](#) · 35 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>plugins { id 'java' }</code>	Configura esta propriedade ou tarefa do Gradle: plugins { id 'java' }.
2	<code>group = 'br.ufsc.sma'</code>	Configura esta propriedade ou tarefa do Gradle: group = 'br.ufsc.sma'.
3	<code>version = '0.1.0'</code>	Configura esta propriedade ou tarefa do Gradle: version = '0.1.0'.
4	<code>defaultTasks 'run'</code>	Configura esta propriedade ou tarefa do Gradle: defaultTasks 'run'.
5	<code>repositories {</code>	Configura esta propriedade ou tarefa do Gradle: repositories.
6	<code> maven { url 'https://raw.githubusercontent.com/jacamo- lang/mvn-repo/master' }</code>	Configura esta propriedade ou tarefa do Gradle: maven { url 'https://raw.githubusercontent.com/jacamo-lang/mvn-repo/master' }.
7	<code> maven { url 'https://repo.gradle.org/gradle/libs-releases' }</code>	Configura esta propriedade ou tarefa do Gradle: maven { url 'https://repo.gradle.org/gradle/libs-releases' }.
8	<code> mavenCentral()</code>	Configura esta propriedade ou tarefa do Gradle: mavenCentral().
9	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
10	<code>dependencies {</code>	Configura esta propriedade ou tarefa do Gradle: dependencies.
11	<code> implementation 'org.jacamo:jacamo:1.3.0'</code>	Adiciona a dependência indicada ao programa Java. A biblioteca local é obtida pelo lançador antes da compilação.
12	<code> implementation files('lib/search.jar')</code>	Adiciona a dependência indicada ao programa Java. A biblioteca local é obtida pelo lançador antes da compilação.
13	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
14	<code>java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }</code>	Configura esta propriedade ou tarefa do Gradle: java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }.
15	<code>sourceSets {</code>	Configura esta propriedade ou tarefa do Gradle: sourceSets.
16	<code> main.java.srcDirs = ['src/env', 'src/agt']</code>	Informa em quais pastas o Gradle deve procurar as fontes desta parte do projeto.
17	<code> test.java.srcDirs = ['src/test/java']</code>	Informa em quais pastas o Gradle deve procurar as fontes desta parte do projeto.
18	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

Linha	Código	Como ler
19	<code>tasks.withType(JavaCompile).configureEach { options.encoding = 'UTF-8' }</code>	Configura esta propriedade ou tarefa do Gradle: <code>tasks.withType(JavaCompile).configureEach { options.encoding = 'UTF-8' }</code> .
20	<code>tasks.register('run', JavaExec) {</code>	Configura esta propriedade ou tarefa do Gradle: <code>tasks.register('run', JavaExec)</code> .
21	<code>dependsOn classes</code>	Exige que a tarefa indicada termine antes de executar esta etapa.
22	<code>mainClass = 'jacamo.infra.JaCaMoLauncher'</code>	Configura esta propriedade ou tarefa do Gradle: <code>mainClass = 'jacamo.infra.JaCaMoLauncher'</code> .
23	<code>classpath = sourceSets.main.runtimeClasspath</code>	Configura esta propriedade ou tarefa do Gradle: <code>classpath = sourceSets.main.runtimeClasspath</code> .
24	<code>args(project.findProperty('jcm') ?: 'gold_miners.jcm')</code>	Configura esta propriedade ou tarefa do Gradle: <code>args(project.findProperty('jcm') ?: 'gold_miners.jcm')</code> .
25	<code>systemProperty 'goldminers.headless', project.findProperty('headless') ?: 'true'</code>	Passa uma propriedade ao processo Java para configurar codificação ou visualização.
26	<code>systemProperty 'java.awt.headless', project.findProperty('headless') ?: 'true'</code>	Passa uma propriedade ao processo Java para configurar codificação ou visualização.
27	<code>systemProperty 'file.encoding', 'UTF-8'</code>	Passa uma propriedade ao processo Java para configurar codificação ou visualização.
28	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
29	<code>tasks.register('runtimeClasspath') {</code>	Configura esta propriedade ou tarefa do Gradle: <code>tasks.register('runtimeClasspath')</code> .
30	<code>dependsOn testClasses</code>	Exige que a tarefa indicada termine antes de executar esta etapa.
31	<code>doLast {</code>	Configura esta propriedade ou tarefa do Gradle: <code>doLast</code> .
32	<code>file('build/runtime-classpath.txt').text = sourceSets.main.runtimeClasspath.asPath</code>	Configura esta propriedade ou tarefa do Gradle: <code>file('build/runtime-classpath.txt').text = sourceSets.main.runtimeClasspath.asPath</code> .
33	<code>file('build/test-runtime- classpath.txt').text = sourceSets.test.runtimeClasspath.asPath</code>	Configura esta propriedade ou tarefa do Gradle: <code>file('build/test-runtime-classpath.txt').text = sourceSets.test.runtimeClasspath.asPath</code> .
34	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
35	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

settings.gradle

Define o nome do projeto usado pelo Gradle.

[Abrir arquivo original](#) · 1 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>rootProject.name = 'gold-miners-sma'</code>	Define ou atualiza <code>rootProject.name</code> com a expressão à direita. Essa informação participa da rotina de preparação deste arquivo.

gold_miners.jcm

Cria o conjunto inicial: líder, um minerador e três agentes inativos, cada minerador com seu artefato de percepção.

[Abrir arquivo original](#) · 38 linhas · numeração da cópia documentada

Linha	Código	Como ler
13	<code>mas gold_miners {</code>	Declara ou delimita a configuração inicial do sistema multiagente.
14	<code>agent leader</code>	Cria o agente indicado e associa seu programa AgentSpeak, quando especificado.
15	<code>agent miner1 : miner.asl {</code>	Cria o agente indicado e associa seu programa AgentSpeak, quando especificado.
16	<code>focus: mining.m1view</code>	Coloca o agente em foco nos artefatos listados para perceber propriedades e executar operações.
17	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
18	<code>agent miner2 : dummy.asl {</code>	Cria o agente indicado e associa seu programa AgentSpeak, quando especificado.
19	<code>focus: mining.m2view</code>	Coloca o agente em foco nos artefatos listados para perceber propriedades e executar operações.
20	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
21	<code>agent miner3 : dummy.asl {</code>	Cria o agente indicado e associa seu programa AgentSpeak, quando especificado.
22	<code>focus: mining.m3view</code>	Coloca o agente em foco nos artefatos listados para perceber propriedades e executar operações.
23	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
24	<code>agent miner4 : dummy.asl {</code>	Cria o agente indicado e associa seu programa AgentSpeak, quando especificado.
25	<code>focus: mining.m4view</code>	Coloca o agente em foco nos artefatos listados para perceber propriedades e executar operações.
26	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
28	<code>workspace mining {</code>	Abre o espaço de trabalho que reúne os artefatos do ambiente.
32	<code>artifact m1view: mining.MiningPlanet(1,0)</code>	Cria o artefato da classe indicada. Os parâmetros selecionam cenário e identificador do minerador quando usados por MiningPlanet.
33	<code>artifact m2view: mining.MiningPlanet(1,1)</code>	Cria o artefato da classe indicada. Os parâmetros selecionam cenário e identificador do minerador quando usados por MiningPlanet.
34	<code>artifact m3view: mining.MiningPlanet(1,2)</code>	Cria o artefato da classe indicada. Os parâmetros selecionam cenário e identificador do minerador quando usados por MiningPlanet.
35	<code>artifact m4view: mining.MiningPlanet(1,3)</code>	Cria o artefato da classe indicada. Os parâmetros selecionam cenário e identificador do minerador quando usados por MiningPlanet.

Linha	Código	Como ler
36	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
37	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

configuracoes/verificacao.jcm

Cria um agente e dois artefatos para conferir uma única entrega. Não configura uma partida competitiva.

[Abrir arquivo original](#) · 9 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	mas verificacao_gold_miners {	Declara ou delimita a configuração inicial do sistema multiagente.
2	agent auditor: src/test/agt/verificacao.asl {	Cria o agente indicado e associa seu programa AgentSpeak, quando especificado.
3	focus: mining.mlview, mining.probe	Coloca o agente em foco nos artefatos listados para perceber propriedades e executar operações.
4	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
5	workspace mining {	Abre o espaço de trabalho que reúne os artefatos do ambiente.
6	artifact mlview: mining.MiningPlanet(3,0)	Cria o artefato da classe indicada. Os parâmetros selecionam cenário e identificador do minerador quando usados por MiningPlanet.
7	artifact probe: mining.ValidationProbe	Cria o artefato da classe indicada. Os parâmetros selecionam cenário e identificador do minerador quando usados por MiningPlanet.
8	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
9	}	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

laboratorio.cmd

Atalho Windows. Entra na pasta correta, escolhe o Python local quando disponível e repassa os argumentos.

[Abrir arquivo original](#) · 9 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	@echo off	Evita repetir cada comando do atalho Windows na tela.
2	setlocal	Mantém as variáveis do atalho restritas a esta execução.
3	cd /d "%~dp0"	Entra na pasta do próprio atalho, inclusive quando ela está em outra unidade.
4	if exist "%~dp0..\..\..\venv-documentos\Scripts\python.exe" (	Só executa este ramo se a condição indicada for satisfeita. O contexto desta rotina é preparação deste arquivo.

Linha	Código	Como ler
5	<code>"%~dp0\..\..\envs\documents\Scripts\python.exe" -X utf8 scripts\laboratorio.py %*</code>	Escolhe o interpretador Python e repassa os argumentos do usuário; o atalho termina com o código devolvido pela execução.
6	<code>) else (</code>	Escolhe o interpretador Python e repassa os argumentos do usuário; o atalho termina com o código devolvido pela execução.
7	<code>python -X utf8 scripts\laboratorio.py %*</code>	Escolhe o interpretador Python e repassa os argumentos do usuário; o atalho termina com o código devolvido pela execução.
8	<code>)</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
9	<code>exit /b %errorlevel%</code>	Devolve ao terminal o código de encerramento do comando executado.

.devcontainer/devcontainer.json

Prepara Java e Python para um ambiente de desenvolvimento. A configuração foi incluída, mas o contêiner ainda não foi executado.

[Abrir arquivo original](#) · 9 linhas · numeração da cópia documentada

Linha	Código	Como ler
1	<code>{</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
2	<code>"name": "Gold Miners – JaCaMo",</code>	Define a opção do contêiner indicada pela chave: nome, imagem, recurso Python, comando de preparação ou usuário de desenvolvimento.
3	<code>"image": "mcr.microsoft.com/devcontainers/java:21-bookworm",</code>	Define a opção do contêiner indicada pela chave: nome, imagem, recurso Python, comando de preparação ou usuário de desenvolvimento.
4	<code>"features": {</code>	Define a opção do contêiner indicada pela chave: nome, imagem, recurso Python, comando de preparação ou usuário de desenvolvimento.
5	<code>"ghcr.io/devcontainers/features/python:1": { "version": "3.12" }</code>	Define a opção do contêiner indicada pela chave: nome, imagem, recurso Python, comando de preparação ou usuário de desenvolvimento.
6	<code>},</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.
7	<code>"postCreateCommand": "python3 scripts/laboratorio.py compilar",</code>	Define a opção do contêiner indicada pela chave: nome, imagem, recurso Python, comando de preparação ou usuário de desenvolvimento.
8	<code>"remoteUser": "vscode"</code>	Define a opção do contêiner indicada pela chave: nome, imagem, recurso Python, comando de preparação ou usuário de desenvolvimento.
9	<code>}</code>	Fecha ou delimita o bloco/estrutura iniciado nas linhas anteriores.

logging.properties

Direciona os registros ao console. O lançador guarda as saídas em arquivos específicos de cada sessão.

Linha	Código	Como ler
7	<code>handlers = java.util.logging.ConsoleHandler</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
20	<code>.level = INFO</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
28	<code>jason.runtime.MASConsoleLogHandler.level = ALL</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
29	<code>jason.runtime.MASConsoleLogHandler.formatter = jason.runtime.MASConsoleLogFormatter</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
31	<code>jason.runtime.MASConsoleLogHandler.tabbed = true</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
32	<code>jason.runtime.MASConsoleLogHandler.colors = true</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
35	<code>java.util.logging.FileHandler.pattern = log/mas-%g.log</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
37	<code>java.util.logging.FileHandler.limit = 500000</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
38	<code>java.util.logging.FileHandler.count = 10</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
39	<code>java.util.logging.FileHandler.formatter = jason.runtime.MASConsoleLogFormatter</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
43	<code>java.util.logging.ConsoleHandler.level = FINE</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
44	<code>java.util.logging.ConsoleHandler.formatter = jason.runtime.MASConsoleLogFormatter</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
46	<code>java.level=OFF</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
47	<code>javax.level=OFF</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
48	<code>sun.level=OFF</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.
49	<code>jade.level=OFF</code>	Configura o registro Java. Linhas sem # são propriedades ativas; os nomes identificam manipulador, formato ou nível.